

Deep Learning-Based Medical Image Classification

Shean De Fonseka

Monash University

Table of Contents

<i>1. Introduction</i>	3
<i>2. Project Objective</i>	4
<i>3. Exploratory Data Analysis (EDA)</i>	4
<i>4. Data Preparation & Processing</i>	9
<i>5. ResNet</i>	10
<i>5.1 ResNet Background</i>	10
<i>5.2 ResNet Architecture</i>	10
<i>5.3 ResNet Forward Propagation</i>	13
<i>5.4 ResNet Back Propagation (Local)</i>	14
<i>5.5 ResNet Back Propagation (Recursively)</i>	15
<i>5.6 ResNet Vanishing Gradient</i>	15
<i>5.7 ResNet Model Integration</i>	17
<i>6. Implementation Details</i>	18
<i>6.1 Loading and Cleaning the Dataset</i>	18
<i>6.2 Extract Unique Labels & Compute Positive Weights (for Weighted BCE Loss)</i>	19
<i>6.3 Read Train / Val and Test Splits</i>	19
<i>6.4 Transformations on Dataset</i>	20
<i>6.5 Creating Data Loaders for Training and Testing</i>	20
<i>6.6 ResNet Model</i>	23
<i>6.7 Loss Function</i>	23
<i>6.8 Optimizer (Adam)</i>	25
<i>7. Results & Evaluation</i>	29
<i>7.1 Confusion Matrix Theory</i>	29
<i>7.2 ROC Curve Theory</i>	30
<i>7.3 Results</i>	32

8. Conclusion	35
9. Appendix	36
9.1 Derivative Proof	36
9.2 Generalized Formula	36
9.3 What is a norm?	37
9.4 What happens if Wl is < 1 ? (vanishing gradient)	37
9.5 What happens if Wl is > 1 ? (exploding gradient)	37
9.6 Forward Propagation Example	38
9.7 Images as Tensor	38
9.8 Convolution Layer	38
9.9 Pooling Operation	41
9.10 Covariate Shift & Internal Covariate Shift	43
9.11 Batch Normalization	44
9.12 Linear Layer	45
9.13 Activation Function (ReLU)	46
9.14 Bounding Box Algorithm	46
9.15 Model Underfitting vs Overfitting	50
9.16 Hyperparameters Used	51
9.17 BCE Loss Example	51
10. Glossary	53
11. References	54

Abstract

Chest X-ray (CXR) analysis is an important for diagnosing thoracic diseases. In this study, we implement a deep learning-based approach integrating the ResNet18 architecture from scratch to classify CXR images into 14 thoracic disease categories using a large-scale dataset from the National Institutes of Health (NIH) Clinical Center.

To enhance model performance, we address class imbalance by reducing the "No Finding" category and applying a weighted Binary Cross-Entropy (BCE) loss. Additionally, we incorporate interpretability by extracting the last feature map from ResNet, highlighting the key regions influencing model predictions. These highlighted areas are enclosed in bounding boxes to provide visual justification for disease classification. The model's performance is evaluated using the Receiver Operating Characteristic (ROC) curve and Area Under the Curve (AUC) metrics.

Our results demonstrate significant improvements over prior benchmark models, outperforming the original academic paper's ResNet-50 model in most disease categories. Notably, the integrated model achieves higher AUC scores for Cardiomegaly (87.32% vs. 81.41%), Effusion (80.33% vs. 73.62%), and Infiltration (67.39% vs. 61.28%), among others. These findings underscore the effectiveness of the proposed approach in improving disease classification accuracy while providing interpretable insights into model decisions.

Keywords: Computer Vision, Residual Networks (ResNet)

1. Introduction

Chest X-ray (*Glossary*), CXR, analysis is an important in medical imaging for diagnosing a wide range of thoracic diseases (*Glossary*). Recent advances in deep learning have enabled automated classification of chest X-rays, offering potential improvements in diagnostic accuracy, efficiency, and accessibility. However, accurate disease classification in CXR remains a challenging task due to several factors: the complexity of disease patterns, the presence of overlapping features, and the inherent variability in image quality. In this study, we utilize a deep learning-based approach to classify CXR images into 14 disease categories, utilizing a large-scale dataset extracted from the National Institutes of Health (NIH) Clinical Center.

The dataset consists of 112,120 frontal-view CXR images, each labeled with one or more of the 14 disease conditions: Atelectasis, Cardiomegaly, Effusion, Infiltration, Mass, Nodule, Pneumonia, Pneumothorax, Consolidation, Edema, Emphysema, Fibrosis, Pleural Thickening, and Hernia (*Glossary*). The images are provided in PNG format with a resolution of 1024x1024 pixels, approximately 45GB in total size. These images are systematically categorized under the "images_0XX" folder structure, with an accompanying metadata file, `Data_Entry_2017.csv`, linking each image to its corresponding disease labels. Additional attributes such as patient ID, age, gender, and imaging parameters are included in the metadata but are not utilized in our study, as we focus solely on image-based disease classification.

The dataset is split into training/validation and test sets at the patient level, preventing data leakage (*Glossary*) by ensuring that all studies from a single patient appear exclusively in one subset. Furthermore, the dataset includes `BBox_List_2017.csv`, containing bounding box annotations for approximately 1,000 images, but these annotations are excluded in this study as we do not employ object detection techniques like R-CNN.

The disease labels for each image were extracted using natural language processing (NLP) from radiological (*Glossary*) reports, achieving an estimated accuracy exceeding 90%. However, given the limitations of entity extraction and the variability in radiologists' descriptions, some labels may exhibit inherent uncertainties. Notably, the 'No finding' label indicates the absence of any of the 14 predefined disease patterns, though it does not necessarily imply a normal chest X-ray.

Given the large-scale nature of the dataset and computational constraints, we employ Kaggle's cloud-based environment for model training, utilizing the GPU T4 x 2 configuration. This choice is motivated by the availability of pre-loaded datasets, facilitating seamless data access and extraction without requiring local storage. Our study aims to investigate the effectiveness of a pure ResNet model in classifying thoracic diseases from CXR images, evaluating its performance on the NIH dataset while addressing challenges associated with multi-label classification and dataset imbalance.

2. Project Objective

The objective of this project is to develop an automated system for the classification of thoracic diseases from Chest X-ray (CXR) images using a deep learning-based approach. Specifically, we integrate a ResNet model to classify 14 distinct thoracic diseases from the dataset.

Beyond classification, we extend our approach to enhance interpretability by extracting the final feature map from the ResNet model. This feature map highlights the regions of interest that contribute most significantly to the classification decision. To further support the model's predictions, we utilize this feature map as part of an object detection mechanism, enclosing the identified regions in bounding boxes. This approach provides a visual justification for the model's classification choices, offering insights into the decision-making process.

To assess the model's performance, we employ the Receiver Operating Characteristic (ROC) curve and compute the Area Under the Curve (AUC) for each disease class. This evaluation metric aligns with the methodology proposed in the original 'ChestX-ray8 Weekly Supervised Classification' paper and serves as a measure of classification effectiveness.

3. Exploratory Data Analysis (EDA)

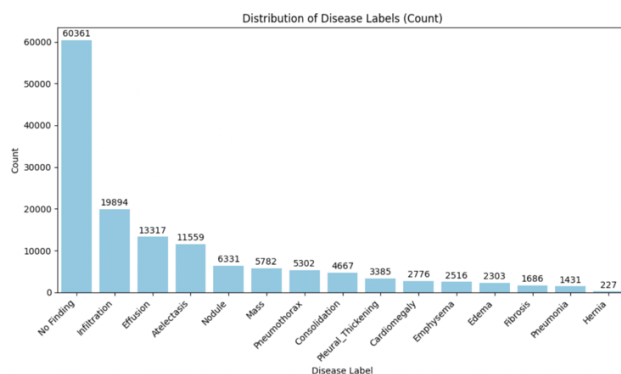


Figure 1: Distribution of Disease Labels

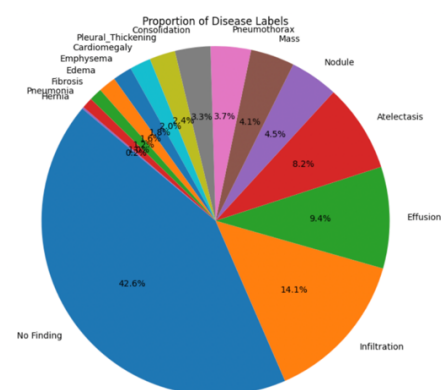


Figure 2: Proportion of Diseases

Class Imbalance:

- "No Finding" is the most frequent category with 60,361 occurrences, which is significantly higher than any other disease label.
- This suggests a strong class imbalance, which could pose challenges in training a model (e.g., bias towards "No Finding").

Top Disease Labels:

- "Infiltration" (19,894 occurrences) is the most common abnormality, followed by "Effusion" (13,317) and "Atelectasis" (11,559).
- These three categories have a much higher prevalence compared to others.

Rare Disease Labels:

- "Hernia" (227 occurrences) is the least common label.
- Other less frequent diseases include "Pneumonia" (1,431), "Fibrosis" (1,686), and "Edema" (2,303).
- Such rare categories might suffer from underrepresentation, making it difficult for a model to learn patterns for these diseases.

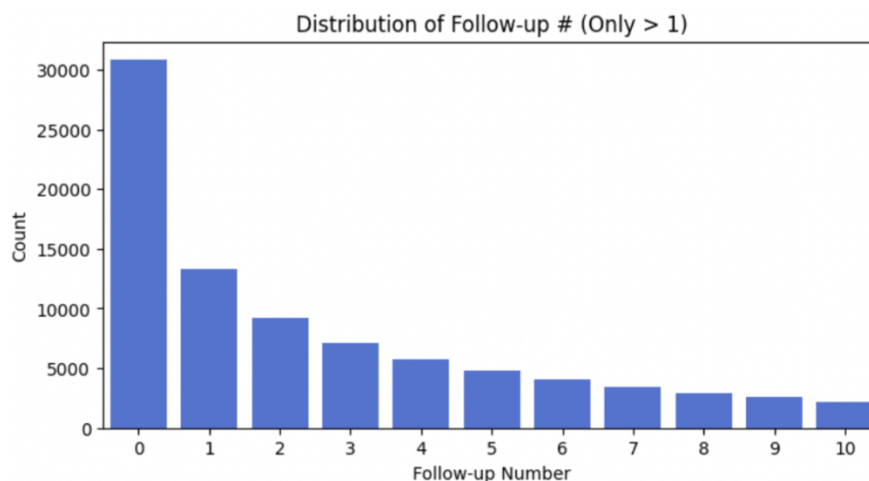


Figure 3: Distribution of Follow Up #

The distribution of follow-up numbers for chest X-rays. The x-axis shows the number of follow-ups (visits to the doctor for additional chest X-rays), while the y-axis represents the count of patients with each respective follow-up number.

Most Patients Have No Follow-up (0 Follow-ups)

- The majority of patients (~30,000 cases) do not return for a follow-up X-ray.
- This suggests that many initial X-ray results were either normal or did not require further examination.

Sharp Drop-off After the First Follow-up

- There is a significant decrease in patient count after 1 follow-up, and the trend continues to decline as the follow-up number increases.

- This indicates that repeated follow-ups are relatively rare, with very few patients requiring more than 5 follow-ups.

Long-Tail Distribution

- While most patients have between 0 to 3 follow-ups, a small percentage require 6-10 follow-ups.
- These cases likely involve chronic or severe conditions that require continuous monitoring over time.

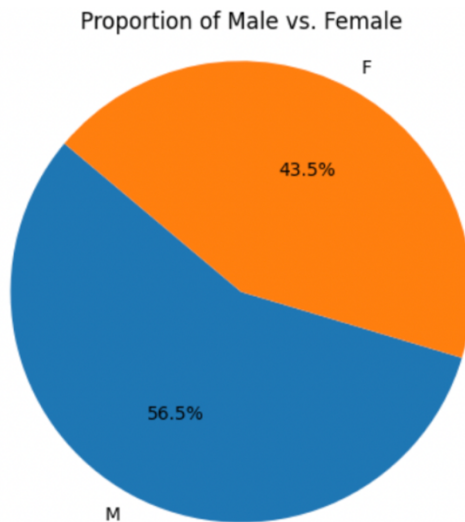


Figure 4: Proportion of Male vs. Female

Slight Male Dominance:

- Males (M) make up 56.5% of the dataset.
- Females (F) account for 43.5%.
- While the dataset is somewhat balanced, there is a slight male majority.

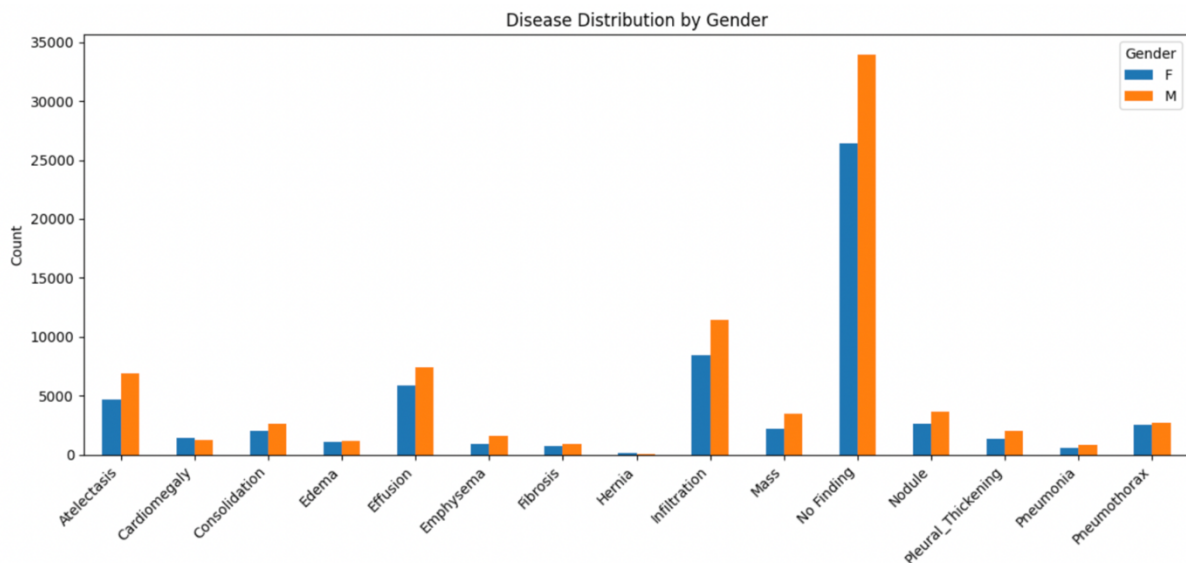


Figure 5: Disease Distribution by Gender

The distribution of various diseases based on gender (Male vs. Female). The x-axis represents different disease labels, while the y-axis represents the count of cases for each disease. The colors differentiate between Males (orange) and Females (blue).

"No Finding" is the Most Common Label:

- Both males and females have a large number of cases with "No Finding."
- However, males have more "No Finding" cases than females, which aligns with the slight male majority (56.5%) in the dataset.

Males Have Higher Counts for Most Diseases:

- Infiltration, Effusion, Cardiomegaly, and Mass show noticeably higher counts in males.
- Conditions like Pneumothorax, Emphysema, and Pleural Thickening are also more frequent in males.

Diseases with Relatively Similar Distribution:

- Diseases like Atelectasis, Pneumonia, and Consolidation appear to have less gender disparity, with similar case counts for both males and females.

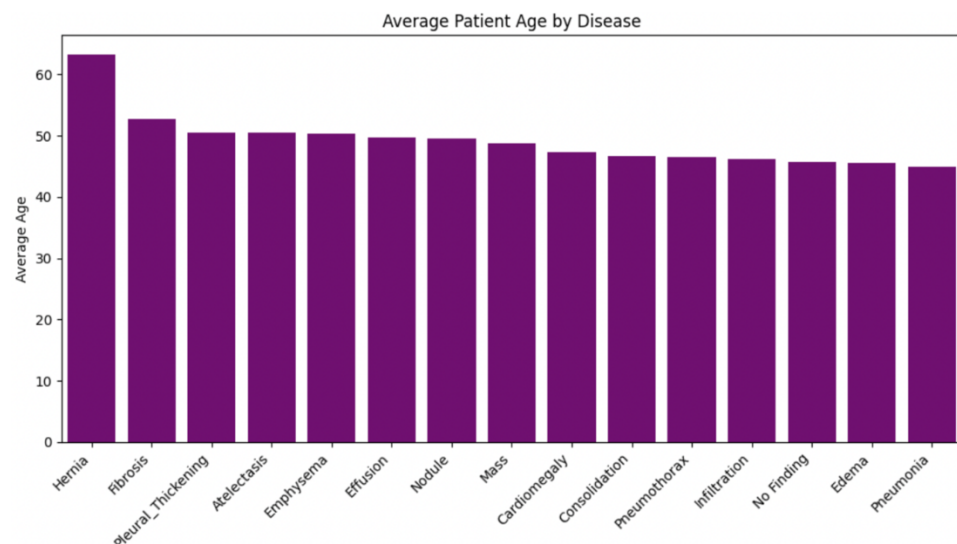


Figure 6: Average Patient Age by Disease

The average patient age for different diseases. The x-axis lists various diseases, while the y-axis shows the average age of patients diagnosed with each condition.

Hernia Has the Highest Average Age:

- Hernia patients have the highest average age (~63 years old), suggesting that this condition is more prevalent in older individuals.
- This aligns with medical expectations, as hernias are often associated with weakened muscles due to aging.

Fibrosis, Pleural Thickening, and Atelectasis Affect Older Patients:

- These conditions have an average patient age above 50, indicating they are more common in elderly individuals.

Conditions with Lower Average Ages (~45-50 years):

- Diseases like Pneumothorax, Pneumonia, and Edema have slightly lower average ages compared to other conditions.

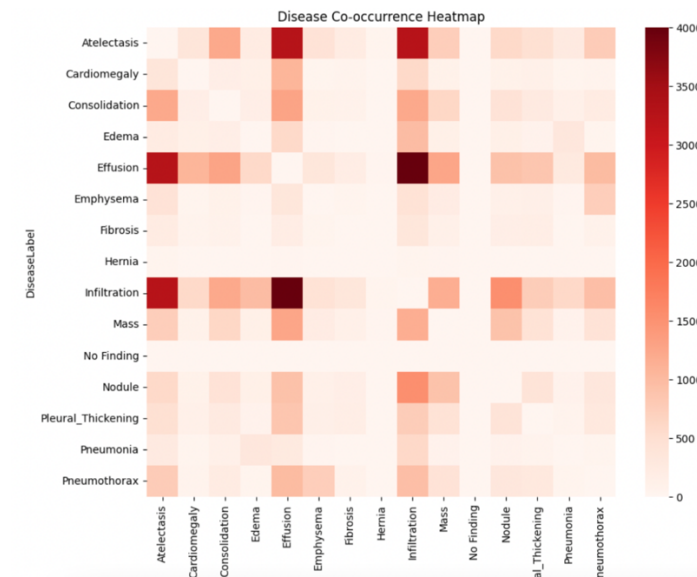


Figure 7: Disease Co-occurrence Heatmap

This heatmap represents how frequently different diseases co-occur in the dataset. Each cell's intensity (color) indicates the number of cases where two diseases appear together in the same patient. Darker shades represent higher co-occurrence.

Effusion and Infiltration Have the Highest Co-occurrence:

- These two diseases show the darkest cells, indicating they frequently appear together.
- Effusion (fluid in the lungs) and Infiltration (substances entering lung tissue) often co-exist.

Atelectasis is Strongly Associated with Effusion:

- Atelectasis (lung collapse) and Effusion (fluid accumulation) commonly appear together.
- This makes sense medically, as fluid buildup in the lungs can cause partial lung collapse.

Some Diseases Have Very Low Co-occurrence:

- Hernia, Pneumonia, and Fibrosis appear in light-colored regions, indicating that they rarely co-occur with other diseases.
- This suggests that these conditions might be more isolated diagnoses rather than occurring alongside other lung-related issues.

Descriptive Statistics:		
	Patient Age	Follow-up #
count	112120.000000	112120.000000
mean	46.901463	8.573751
std	16.839923	15.406320
min	1.000000	0.000000
25%	35.000000	0.000000
50%	49.000000	3.000000
75%	59.000000	10.000000
max	414.000000	183.000000

Figure 8: Descriptive Statistics of Patient Age and Follow Up

For reference, these are the only two variables in which we can logically derive descriptive statistics. All other values are either not numerical in value or don't represent a distribution.

Patient Age:

- Mean (Average Age): 46.9 years. The dataset primarily consists of middle-aged patients.
- Standard Deviation (std): 16.84 years: There is a moderate spread in ages, suggesting a mix of younger and older patients.
- Minimum Age: 1 year. Indicates that some patients are infants.
- Maximum Age: 414 years. This is clearly an outlier or data entry error, as no human has lived that long.
 - This is removed as explained later on as we consider extreme outliers' faulty entries in the dataset.

Follow-up Number:

- Mean (Average Follow-ups): 8.57. On average, patients have around 9 follow-up visits.
- Standard Deviation: 15.4. Follow-up numbers vary widely.
- Minimum Follow-ups: 0. Some patients do not return for additional X-rays.
- Maximum Follow-ups: 183. Some patients have an extremely high number of follow-ups, possibly due to chronic conditions.
 - This is removed as explained later. We consider this an extreme outlier and only take reasonable visits.

4. Data Preparation & Processing

Given the imbalance in the dataset across classes, we decided to reduce the number of 'No Finding' chest X-ray images by half in the training dataset. This approach retains a substantial number of "No Finding" samples while mitigating the class imbalance.

To further address class imbalance, we incorporated a weighted binary cross-entropy (BCE) loss function, integrating a `pos_weight` tensor of size equal to the number of classes. This adjustment increases the penalty for misclassifications in the positive (minority) class, ensuring that the network assigns greater attention to these underrepresented examples. This is discussed in detail in later sections.

Additionally, we identified and removed faulty data entries. Some rows contained implausible values, such as ages exceeding 200 years and follow-up numbers greater than 100. While a high follow-up count could be possible, these extreme values were treated as outliers and removed to maintain the integrity of the dataset. Descriptive statistical analysis was used to detect and filter these anomalies, ensuring that our dataset remains representative of common clinical cases.

5. ResNet

5.1 ResNet Background

The ResNet (Residual Network) architecture was introduced by Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun from Microsoft Research Asia. It was presented at the CVPR 2016 conference, the ResNet model tackled the challenges of training very deep networks, particularly the issue of vanishing gradients, which often prevented the stability of models with more than 30 layers.

ResNet introduced skip (residual) connections that allow the network to learn residual mappings instead of complete transformations. These connections enable the network to adjust feature representations incrementally, rather than having to learn entirely new feature transformations. This significantly improves gradient flow, stabilizes training, and allows for deeper architectures without degradation in performance.

ResNet's dominated the 2015 ImageNet Large Scale Visual Recognition Challenge (ILSVRC) with state-of-the-art performance. The modular architecture consists of basic blocks (used in ResNet-18 and ResNet-34) and bottleneck blocks (used in ResNet-50, ResNet-101, and ResNet-152), which allow the network to scale efficiently. This modular design facilitated widespread adoption and adaptation by the research community, enabling further advancements in deep learning applications.

5.2 ResNet Architecture

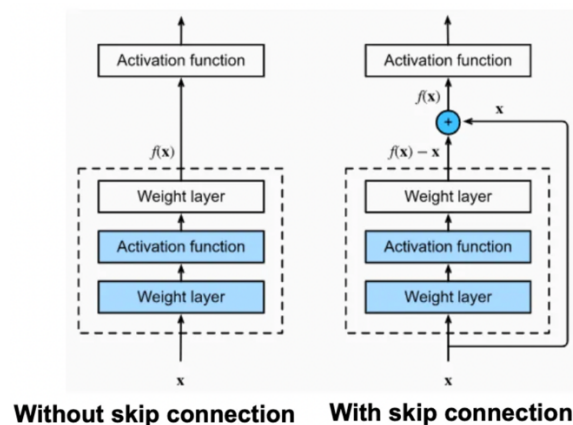


Figure 9: ResNet Residual Block (Zhang, 2023)

In a standard convolutional neural network (without skip connections), an input image is passed through multiple convolutional layers to extract features from scratch. This approach requires the network to learn a direct transformation from the input space to the output space, which can lead to vanishing gradients and difficulty in training deeper models.

ResNet addresses this issue by introducing skip connections, which allow the model to add the original input back to the transformation applied by the network. Instead of learning the full transformation, the network now learns only the residual function (the changes), where:

$$f(x) = g(x) + x$$

where:

- x is the input image,
- $g(x)$ represents the transformations applied by the convolutional layers,
- $f(x)$ is the final output.

Why Use Skip Connections?

Skip connections significantly improve training efficiency because:

- Learning small differences, $g(x)$, is easier than learning the entire transformation $f(x)$ from scratch.
- If the best transformation is simply x , then $g(x) = 0$, meaning the network only copies x through, reducing the risk of degradation.
- When only minor feature modifications are needed, $g(x)$ learns these small changes, making optimization easier.

Real-World Analogy

Consider the task of revising an essay:

- Without skip connections: You start with a blank page and rewrite the entire essay from scratch.
- With skip connections (ResNet): You start with an existing draft and make only the necessary edits.

Let's now review the building blocks of the ResNet model. Below is the Residual component.

```
class Residual(nn.Module):
    def __init__(self, num_channels, use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.LazyConv2d(num_channels, kernel_size=3, stride=strides, padding=1)
        self.conv2 = nn.LazyConv2d(num_channels, kernel_size=3, padding=1)
        self.conv3 = None
        if use_1x1conv:
            self.conv3 = nn.LazyConv2d(num_channels, kernel_size=1, stride=strides, padding=1)
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.bn2 = nn.BatchNorm2d(num_channels)
        self.relu = nn.ReLU()

    def forward(self, X):
        Y = self.relu(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3 is not None:
            X = self.conv3(X)
        Y += X
        return self.relu(Y)
```

Figure 10: ResNet Residual Code

It is important to note that x is being passed through the residual block in the forward pass, so x is always added to $g(x)$. This ResNet model is always under the assumption of learning the difference between $g(x)$ and x .

What is the point of the use_1x1conv?

- It can handle the shape changes to ensure that x back to $g(x)$, even when the dimensions don't match.
- In other words, we apply the skip connection when the dimensions of x (the input) and $g(x)$ (the transformed output) do not match.

It is also important to remember and note that the dimensions of the input is:

$$[batch_size, channels, height, width]$$

ResNet follows the same principle, rather than forcing the network to learn a completely new feature map, it allows the network to refine existing features.

```
class ResnetBlock(nn.Module):
    def __init__(self, num_channels, num_residuals, first_block=False, **kwargs):
        super(ResnetBlock, self).__init__(**kwargs)
        self.residual_layers = []
        for i in range(num_residuals):
            if i == 0 and not first_block:
                self.residual_layers.append(
                    Residual(num_channels, use_1x1conv=True, strides=2))
            else:
                self.residual_layers.append(Residual(num_channels))
        self.residual_blk = nn.ModuleList(self.residual_layers)

    def forward(self, X):
        for layer in self.residual_blk:
            X = layer(X)
        return X
```

Figure 11: ResNet Block Code

A ResNet block is defined as consisting of several residual blocks.

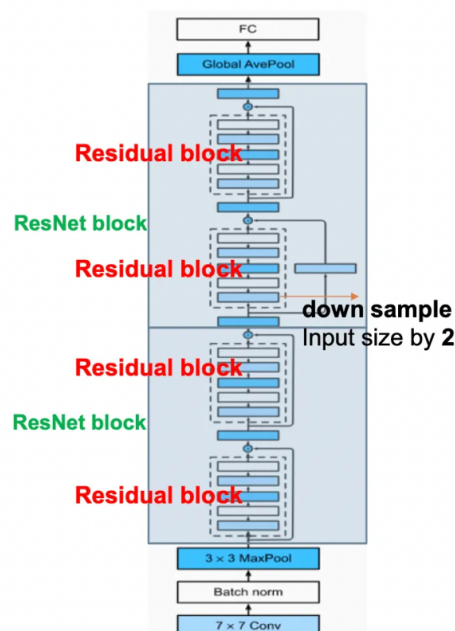


Figure 12: ResNet18 Architecture (Zhang, 2023)


```

class create_ResNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.ModuleList([
            nn.LazyConv2d(64, kernel_size=7, stride=2, padding=3),
            nn.LazyBatchNorm2d(),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            ResnetBlock(64, 2, first_block=True),
            ResnetBlock(128, 2),
            ResnetBlock(256, 2),
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(1),
            nn.LazyLinear(10),
            # nn.Softmax(dim=-1)
        ])
    def forward(self, X):
        for _, layer in enumerate(self.layers):
            X = layer(X)
        return X

```

Figure 13: ResNet Architecture Code

The first ResNet block receives input that already has 64 channels due to the LazyConv2d transformation (turning [INPUT IMAGE DIMENSION] $\rightarrow$ 64). The dimensions remain the same even for the LazyBatchNorm2d, ReLU and MaxPool2d.

The Residual block processes this input without changing the number of channels.

- The input X is already in the correct shape (64 channels, 56×56 spatial size).
- The Residual block does not change the number of channels in this case.
- So, when we add X back, there is no shape mismatch.

Thus, we can safely add Y and X together.

However, when we increase dimensions in the second ResNet block from 64 to 128, this means that the second ResNet requires an image dimension size of 128. From the previous line, the image dimension size is 64.

This means, the skip connection (X) has different dimensions from Y :

- X has 64 channels, 56×56.
- $g(X)$ has 128 channels, 28×28 (after stride=2 down sampling).

5.3 ResNet Forward Propagation

Instead of just one residual block, let's recursively apply this formula through multiple layers.

Each deeper residual block follows the same principle:

$$g(x_i) = x_i + f(x_i)$$

Applying recursively:

$$g(x_L) = x_1 + \sum_{i=1}^{L-1} f(x_i)$$

which means that at layer x_L , the final output x_L is the sum of:

- The original input x_L (directly propagated).
- The accumulated residuals

$$\sum_{i=l}^{L-1} f(x_i)$$

Any x_1 is an additive outcome. This contrasts with the typical multiplicative gradient in other deep learning models:

$$g(x_L) = \prod_{i=l}^{L-1} W_i x_1$$

Reference *Appendix 9.6 Forward Propagation Example*

5.4 ResNet Back Propagation (Local)

In ResNet, each residual block follows the equation:

$$g(x) = x + f(x)$$

where:

- x is the input to the residual block
- $f(x)$ is the learned residual transformation.

Now, when computing gradients:

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial g(x)} \cdot \frac{\partial g(x)}{\partial x}$$

Since:

$$g(x) = x + f(x)$$

we get:

$$\frac{\partial g(x)}{\partial x} = 1 + \frac{\partial f(x)}{\partial x}$$

Thus, the gradient of x is:

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial g(x)} \cdot (1 + f'(x))$$

Expanding the brackets we get:

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial g(x)} + \frac{\partial E}{\partial g(x)} \cdot f'(x)$$

which means the gradient is an additive sum of two terms:

1. The direct contribution from the next layer.

$$\frac{\partial E}{\partial g(x)}$$

2. The contribution from the learned transformation.

$$\frac{\partial E}{\partial g(x)} \cdot f'(x)$$

This is a local backpropagation formula within one residual block.

5.5 ResNet Back Propagation (Recursively)

To compute the gradient of the loss E with respect to an earlier layer x_l , we use the chain rule:

$$\frac{\partial E}{\partial x_l} = \frac{\partial E}{\partial g(x_L)} \cdot \frac{\partial g(x_L)}{\partial x_l}$$

From our recursive equation:

$$g(x_L) = x_1 + \sum_{i=1}^{L-1} F(x_i)$$

So, differentiating with respect to x_l :

$$\frac{\partial g(x_L)}{\partial x_1} = 1 + \frac{\partial}{\partial x_1} \sum_{i=1}^{L-1} F(x_i)$$

Substituting into the gradient equation:

$$\frac{\partial E}{\partial x_l} = \frac{\partial E}{\partial g(x_L)} \cdot \left(1 + \frac{\partial}{\partial x_l} \sum_{i=1}^{L-1} F(x_i) \right)$$

This is in contrast to multiplicative model in other deep learning models:

$$\frac{\partial E}{\partial g(x_L)} = \prod_{i=1}^{L-1} W_i \frac{\partial E}{\partial g(x_L)}$$

5.6 ResNet Vanishing Gradient

What is the Vanishing Gradient Problem?

Assume we have a network with L layers.

For simplicity, let's take the network to be scalar, with one neuron per layer.

The network is defined as follows:

Input:

$$a_0 = x.$$

For each layer $l = 1, 2, \dots, L$:

$$z_l = W_l \cdot a_{l-1} + b_l,$$

$$a_l = \sigma(z_l),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is the sigmoid function and its derivative is

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Key fact:

- The maximum value of $\sigma'(z)$ is 0.25, and it occurs when $\sigma(z) = 0.5$ (i.e. when $z = 0$).

Choosing Parameters to Illustrate Maximum Vanishing Effect

To illustrate the vanishing gradient phenomenon in its worst case, we can choose the weights and biases so that the pre-activation at each layer is zero. That is, for every layer l :

$$z_l = 0 \Rightarrow a_l = \sigma(0) = 0.5 \text{ and } \sigma'(0) = 0.25.$$

A simple way to achieve this is:

For layer 1:

Choose $W_1 = 1$ and set the bias $b_1 = -x$ so that:

$$z_1 = W_1 \cdot x + b_1 = x - x = 0.$$

For layer 2:

Since $a_1 = 0.5$, choose $W_2 = 1$ and set $b_2 = -0.5$ so that:

$$z_2 = W_2 a_1 + b_2 = 0.5 - 0.5 = 0.$$

Repeat for all layers:

At each layer l , choose $W_l = 1$ and set b_l such that:

$$z_l = 1 \cdot a_{l-1} + b_l = 0 \Rightarrow b_l = -a_{l-1}.$$

Thus, in every layer:

$$a_l = \sigma(0) = 0.5, \sigma'(z_l) = 0.25.$$

Backpropagation and the Chain Rule

When training the network, the gradient of the loss L with respect to the input or an early weight is computed via the chain rule. In a deep network, the derivative of the final output a_L with respect to the input a_0 is:

$$\frac{da_L}{da_0} = \frac{da_L}{dx} = \prod_{l=1}^L \frac{da_l}{da_{l-1}}$$

We apply the chain rule here where each term is

$$\frac{da_l}{da_{l-1}} = \sigma'(z_l) \cdot W_l$$

To get the equation above, refer to *Appendix 9.1 Derivative Proof*.

To get the generalized formula, refer to *Appendix 9.2 Generalized Formula*.

When you chain these derivatives across L layers, the overall gradient from the output back to the input is approximately:

$$\frac{da_L}{da_0} = \frac{da_L}{dx} = \prod_{l=1}^L \sigma'(z_l) \cdot W_l$$

Now, suppose that for every layer, the weight matrix W_l has a norm close to 1. This means that W_l neither significantly amplifies nor shrinks the gradient on its own.

The general formula / overall product:

$$(c \cdot ||W||)^L$$

Reference *Appendix 9.3 What is a norm?* for more information on the norm theory.

Reference *Appendix 9.4 What happens if W_l is < 1 ? (vanishing gradient)*

Reference *Appendix 9.5 What happens if W_l is > 1 ? (exploding gradient)*

Now if $(c \cdot ||W||)^L < 1$, then each layer multiplies the gradient by a factor less than 1. Repeating this over L layers gives:

$$(c \cdot ||W||)^L \rightarrow 0 \text{ as } L \text{ increases}$$

- This means the gradient shrinks exponentially with the number of layers, it vanishes.

Now if $(c \cdot ||W||)^L > 1$, then each layer multiplies the gradient by a factor greater than 1. Repeating this over L layers gives:

$$(c \cdot ||W||)^L \rightarrow \infty \text{ as } L \text{ increases}$$

- and the gradient explodes.

This product shows that even if the weights do not diminish the gradient (they are 1), the repeated multiplication of the derivative of the activation function (which is at most 0.25) causes the gradient to shrink exponentially with the number of layers.

These numbers demonstrate an exponential decay in the gradient as the network depth increases. The gradients arriving at the early layers become so small that they are effectively zero for practical purposes, causing those layers to learn very little during training.

What is special about ResNet model?

In ResNet, we instead have:

$$\nabla f(x) = \nabla g(x) + 1$$

Key Difference: The "+1" ensures that the gradient never completely vanishes!

- Even if $\nabla g(x)$ becomes very small, the gradient still contains a strong signal from the identity mapping.

$$\nabla g(x)$$

This ensures that gradients flow better back to earlier layers, keeping learning stable.

5.7 ResNet Model Integration

It's worth mentioning the following small changes made to the ResNet model we have integrated.

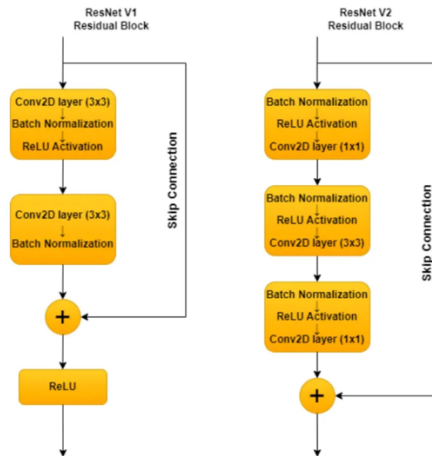


Figure 14: ResNet Residual Block Difference, V1 and V2. (Wyszynski, 2023)

In the original ResNet paper (He et al., 2015/2016), ResNet-18 and ResNet-34 use a basic residual block (the one you’ve shown, consisting of two 3×3 convolutions). This is not the case for ResNet-50, ResNet-101, and ResNet-152 use a bottleneck block with three convolutions:

- 1×1 convolution (reduces dimensionality)
- 3×3 convolution
- 1×1 convolution (restores dimensionality)

The original ResNet (ResNet v1) applied activation after the addition $x + g(x)$ inside each residual block (a.k.a. “post-activation”). Later, ResNet v2 introduced pre-activation: batch normalization + ReLU before the convolution inside each branch.

6. Implementation Details

6.1 Loading and Cleaning the Dataset

```

# Focus on these two columns.
cols = ["Patient Age", "Follow-up #"]

# Calculate IQR bounds for Patient Age
Q1_age = df["Patient Age"].quantile(0.25)
Q3_age = df["Patient Age"].quantile(0.75)
IQR_age = Q3_age - Q1_age
age_lower_bound = Q1_age - 1.5 * IQR_age
age_upper_bound = Q3_age + 1.5 * IQR_age

# Calculate IQR bounds for Follow-up #
Q1_follow = df["Follow-up #"].quantile(0.25)
Q3_follow = df["Follow-up #"].quantile(0.75)
IQR_follow = Q3_follow - Q1_follow
follow_lower_bound = Q1_follow - 1.5 * IQR_follow
follow_upper_bound = Q3_follow + 1.5 * IQR_follow

# Create masks for each column
mask_age = (df["Patient Age"] >= age_lower_bound) & (df["Patient Age"] <= age_upper_bound)
mask_follow = (df["Follow-up #"] >= follow_lower_bound) & (df["Follow-up #"] <= follow_upper_bound)

# Combine the masks (keep only rows passing both filters)
final_mask = mask_age & mask_follow
df_cleaned = df[final_mask].copy()

original_count = df.shape[0]
final_count = df_cleaned.shape[0]
total_removed = original_count - final_count
print("\n--- Outlier Removal Summary ---")
print(f"Original rows: {original_count}, Final rows: {final_count}, Removed: {total_removed}")

```

In the dataset, some of the rows contain Patient Age and Follow Up values which are quite unrealistically large and for safe keepings, we have decided to remove any rows that are outside of the Upper and Lower bounds.

The formulas are:

$$\text{Lower Bound} = Q1 - 1.5 \cdot (IQR)$$

$$\text{Lower Bound} = Q3 + 1.5 \cdot (IQR)$$

$$IQR = Q3 - Q1$$

These questions are done for both Patient Age and Follow Up.

6.2 Extract Unique Labels & Compute Positive Weights (for Weighted BCE Loss)

```
# Extract the unique set of classes from "Finding Labels"
unique_labels = set()
for labels_str in df_cleaned["Finding Labels"]:
    # Split the string by "|" (since an image may have multiple labels)
    labels = labels_str.split("|")
    unique_labels.update(labels)
unique_labels = sorted(unique_labels)
print("Unique labels found:", unique_labels)

# Compute positive counts for each class.
total_images = len(df_cleaned)
pos_counts = {cls: 0 for cls in unique_labels}
for labels_str in df_cleaned["Finding Labels"]:
    labels = labels_str.split("|")
    for cls in unique_labels:
        if cls in labels:
            pos_counts[cls] += 1

# Convert counts to tensors.
pos_counts_tensor = torch.tensor([pos_counts[cls] for cls in unique_labels], dtype=torch.float)
neg_counts_tensor = torch.tensor([total_images - pos_counts[cls] for cls in unique_labels], dtype=torch.float)
pos_weight = neg_counts_tensor / pos_counts_tensor
print("Positive weights:", pos_weight)
```

The first part finds the unique classes (thoracic diseases) under the ‘Finding Labels’ column in the dataset. Given that this column can consist of one or more diseases (separated by a ‘|’), we have to iterate split and update the dictionary accordingly. This ultimately outputs to 15 classes (14 diseases and normal).

The second part is finding the positive weights for the Weighted BCE loss function. To understand this, we must break down the loss theory in detail. This is discussed later below.

6.3 Read Train / Val and Test Splits

```
# Read the file "train_val_list.txt" which contains the filenames (one per line) for the training/validation set.
with open(train_val_list_path, "r") as f:
    # f.read() reads the entire file into a single string.
    # .splitlines() splits the string at line breaks, creating a list where each element is a filename.
    train_val_files = f.read().splitlines()

# Read the file "test_list.txt" which contains the filenames for the test set.
with open(test_list_path, "r") as f:
    # Similar to above, this creates a list of filenames for the test images.
    test_files = f.read().splitlines()

# Filter the metadata DataFrame (df_cleaned) to create a DataFrame for the training/validation set.
# The filtering uses the .isin() method to select only rows where the "Image Index" column's value
# is present in the train_val_files list.
df_train_val = df_cleaned[df_cleaned["Image Index"].isin(train_val_files)]

# Similarly, filter the metadata DataFrame to create a DataFrame for the test set.
df_test = df_cleaned[df_cleaned["Image Index"].isin(test_files)]

print("Train/Val set size:", df_train_val.shape[0])
print("Test set size:", df_test.shape[0])
```

From the dataset, we are given two files named ‘train_val_list.txt’ and ‘test_list.txt’. The train_val_list.txt file contains all the images which are recommended to use for the training followed by validation of the training model. The test_list.txt is used purely for validation purposes after training the model.

The reason for this is that they separated the patients into their respective bins (either in train_val_list.txt or test_list.txt). This prevents data leakage in seeing a patient’s Chest X-ray in the test and then in the test

(which gives it a past reference or future leak). Thus, we train on a group of Patient's Chest X-ray and test on another separate group of Patient's Chest X-ray.

6.4 Transformations on Dataset

```
# Define image transformations. Adjust parameters as needed.
train_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((256, 256)),
    transforms.RandomCrop((224, 224)),
    transforms.RandomRotation(degrees=15),
    transforms.RandomAffine(degrees=10, translate=(0.05, 0.05), scale=(0.95, 1.05)),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

test_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])
```

Chest X-ray images have limited visual diversity compared to natural images, so we apply augmentation to improve generalization.

Data augmentation (such as rotation, width shifting, zoom, brightness adjustments) is used only during training to introduce variation and reduce overfitting.

However, for validation and testing, we need a consistent, standardized dataset to fairly assess the model's generalization ability. If we augmented the validation data, we might introduce distortions that do not match the real-world test distribution. Thus, in validation, we apply only necessary preprocessing transformations (resizing, grayscale conversion, normalization) but no augmentation.

6.5 Creating Data Loaders for Training and Testing

```
# Create Dataset instances

# This instantiation does the following:
# 1. It receives a DataFrame (df_train_val) that contains metadata for each image,
#    including the filename (under "Image Index") and its associated multi-label string (under "Finding Labels").
# 2. It uses 'img_dir' as the base directory, which contains subfolders (images_001, images_002, ..., images_012) where the actual image files are stored.
# 3. It accepts a transform (here, train_transform) that will be applied to each image when it is loaded.
# 4. The 'unique_labels' list is used to create a multi-label binary target vector for each image.

# In the __init__ method of ChestXrayDataset:
# - The DataFrame is reset (dropping the original index) and then filtered using _file_exists,
#   which checks that the corresponding image file exists in one of the subfolders.
# - This ensures that only valid rows (i.e., images that can be loaded) are kept.

# When the DataLoader later calls __getitem__(idx):
# - The dataset loads the image from disk using the filename.
# - The specified transform is applied to the image (resulting in a tensor of shape [1, 224, 224] if grayscale).
# - A target tensor is created by parsing the "Finding Labels" string and setting the appropriate indices to 1.

# Thus, each item in train_val_dataset is a tuple (image_tensor, target_tensor) - what is being outputted
# image_tensor has shape [1, 224, 224] (for a grayscale image),
# and target_tensor has shape [num_classes] (e.g., [15] for a 15-class problem).
train_val_dataset = ChestXrayDataset(df_train_val, img_dir, transform=train_transform, unique_labels=unique_labels)

# Similarly, create a testing dataset instance.
# This instance uses the test DataFrame (df_test) and a test-specific transform (test_transform),
# which might be similar to train_transform but typically does not include random augmentations.
# Each item in test_dataset is also a tuple (image_tensor, target_tensor)
# __getitem__ is called when running the class
# Dimensions are X where X is the number of train/test images and each of them contain ([1, 224, 224], [15])
train_val_dataset = ChestXrayDataset(df_train_val, img_dir, transform=train_transform, unique_labels=unique_labels)
test_dataset = ChestXrayDataset(df_test, img_dir, transform=test_transform, unique_labels=unique_labels)
```

train_val_dataset is the dataset created for training the model.

test_dataset is the dataset created solely for testing the model.

They are passed through ChestXrayDataset class which transforms the images and creates a structure dataset which can be interpreted by the DataLoader function from PyTorch. There are X values in a list (where X is the number of train/test images) and each of these values contains a tuple of (image, label).


```

# Split the full training/validation dataset (train_val_dataset) into two subsets:
# - train_dataset: for training, with length 'train_size'
# - valid_dataset: for validation, with length 'val_size'
# The random_split function randomly partitions the dataset based on the provided lengths.
# The 'generator' parameter is used to set the random seed (42 in this case) for reproducibility.
train_dataset, valid_dataset = random_split(
    train_val_dataset, # The dataset to be split.
    [train_size, val_size], # A list specifying the sizes of the resulting subsets (as defined above)
    generator=torch.Generator().manual_seed(42)
)

print("Training set:")
print(f"\t- Number of samples: {len(train_dataset)}")
print(f"\t- Shape of one sample: {list(train_dataset[0][0].shape)}")
print("Validation set:")
print(f"\t- Number of samples: {len(valid_dataset)}")
print(f"\t- Shape of one sample: {list(valid_dataset[0][0].shape)}")
print("Test set:")
print(f"\t- Number of samples: {len(test_dataset)}")
print(f"\t- Shape of one sample: {list(test_dataset[0][0].shape)}")

# Create DataLoaders for the training, validation, and test datasets.
# For each sample, __getitem__ returns a tuple: (image_tensor, target_tensor)
# - image_tensor has shape [C, H, W]. For our grayscale images with 224x224 size, that's [1, 224, 224].
# - target_tensor has shape [num_classes]. For our 15-class problem, that's [15].
# When the DataLoader stacks these:
# - The images are stacked into a tensor of shape [batch_size, C, H, W].
#   For a batch size of 64, this becomes [64, 1, 224, 224].
# - The targets are stacked into a tensor of shape [batch_size, num_classes].
#   For a batch size of 64, this becomes [64, 15].
# The shuffle=True option for train_loader ensures that the sample indices are randomized each epoch,
# which helps improve training by preventing the model from learning the order of data.
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=64)
test_loader = DataLoader(test_dataset, batch_size=64)

```

We have two sets of images now. The training dataset and the testing dataset. The training dataset will be used to train the model and the testing dataset will be used after to test the model and evaluate the model.

What we are doing at the top of the code is splitting the training dataset further. 83% of the training dataset is used for training the actual model while the remaining 17% is used to validate the model after training. This is split randomly.

We finally integrate the data loaders for each of these datasets. We initialize with a batch size of 64.

In mini-batch training, we group a set of samples (e.g., 64 images) and feed them to the model at once. If the batch size is 64, the model receives a tensor of shape (64,channels,height,width). This means that all 64 images are processed in parallel through the network's layers, taking advantage of GPU vectorization to produce predictions, logits or probabilities, for each image in the batch.

Once the model generates predictions for all 64 samples, we calculate a single scalar loss for the entire batch. Typically, this is done by computing an individual loss (e.g., cross-entropy or BCE) for each sample, then summing or averaging these values. In many deep learning frameworks like PyTorch, the default is to average these per-sample losses across the batch, yielding a single “batch loss” scalar.

- The **optimization problem** in deep learning has the form
 - $\min_{\theta} L(D; \theta) := \frac{1}{N} \sum_{i=1}^N l(x_i, y_i; \theta) = \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i; \theta))$
- Evaluation of the **full gradient** is **expensive**. We want to just **estimate** this gradient
 - Sample a mini-batch $i_1, i_2, \dots, i_b \sim \text{Uni}(\{1, 2, \dots, N\})$ where b is the mini-batch (batch) size.
 - The batch size is usually 32, 64, 128, 256, and so on.
 - Construct $\tilde{L}(\theta) := \frac{1}{b} \sum_{k=1}^b l(x_{i_k}, y_{i_k}; \theta)$ as the average loss of those in the current batch.
 - $E[\nabla_{\theta} \tilde{L}(\theta_t)] = \nabla_{\theta} L(D; \theta_t)$
 - $\nabla_{\theta} \tilde{L}(\theta_t) = \frac{1}{b} \sum_{k=1}^b \nabla_{\theta} l(x_{i_k}, y_{i_k}; \theta_t)$ is **unbiased** estimation of $\nabla_{\theta} L(D; \theta_t)$
 - $O(b)$ compares to $O(N)$.
- The update rule of SGD
 - $\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \tilde{L}(\theta_t)$ with learning rate $\eta_t \propto O(\frac{1}{t})$
 - We use $\nabla_{\theta} \tilde{L}(\theta_t)$ as an **unbiased estimate** of the full gradient $\nabla_{\theta} L(D; \theta)$
 - How to compute $\nabla_{\theta} \tilde{L}(\theta_t)$ efficiently for **deep networks**?

Figure 15: Loss with Batches Maths Monash University (Phung, 2024)

Note: A clear breakdown of Loss is later discussed.

With this single batch loss, we perform backpropagation. During backprop, the gradient is computed based on the contribution of all 32 samples in the batch. This gradient captures how the model parameters should be adjusted to reduce the loss for the entire mini-batch, rather than a single data point.

Finally, we perform one weight update step using an optimizer (in this case Adam) based on the gradient calculated from the batch loss. After this update, we fetch the next batch of samples and repeat the process. Hence, in each iteration, the model's weights are updated once per mini-batch, incorporating the combined error signals from all samples in that batch.

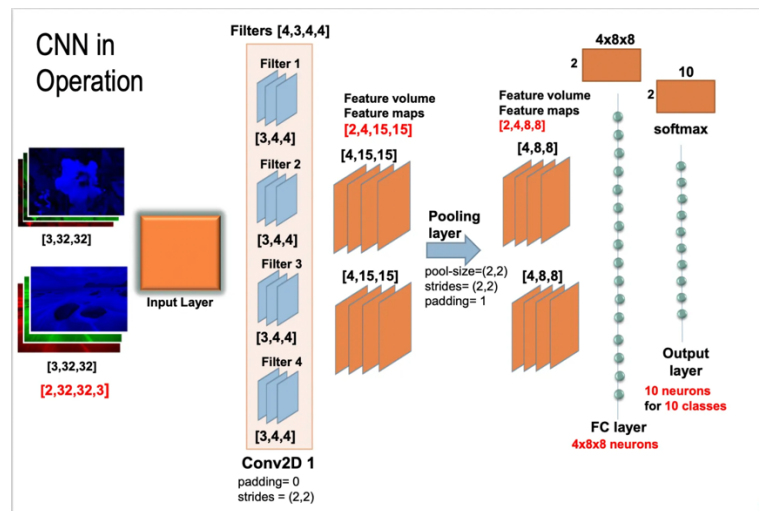


Figure 16: CNN in Operation (Batch Size = 2) Monash University (Phung, 2024)

6.6 ResNet Model

```
# Residual block (skip connection)
class Residual(nn.Module):
    def __init__(self, num_channels, use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.LazyConv2d(num_channels, kernel_size=3, stride=strides, padding=1)
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.relu = nn.ReLU()
        self.conv2 = nn.LazyConv2d(num_channels, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(num_channels)
        self.conv3 = nn.LazyConv2d(num_channels, kernel_size=1, stride=strides) if use_1x1conv else None

    def forward(self, X):
        Y = self.relu(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3 is not None:
            X = self.conv3(X)
        Y += X
        return self.relu(Y)

# ResNet block
class ResnetBlock(nn.Module):
    def __init__(self, num_channels, num_residuals, first_block=False):
        super().__init__()
        layers = []
        for i in range(num_residuals):
            if i == 0 and not first_block:
                layers.append(Residual(num_channels, use_1x1conv=True, strides=2))
            else:
                layers.append(Residual(num_channels))
        self.residual_blk = nn.ModuleList(layers)

    def forward(self, X):
        for layer in self.residual_blk:
            X = layer(X)
        return X

# Complete ResNet model.
class create_ResNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.ModuleList([
            nn.LazyConv2d(64, kernel_size=7, stride=2, padding=3),
            nn.LazyBatchNorm2d(),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            ResnetBlock(64, 2, first_block=True),
            ResnetBlock(128, 2),
            ResnetBlock(256, 2),
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(1),
            nn.LazyLinear(len(unique_labels)) # output dimension = number of classes
        ])
        # For BCEWithLogitsLoss, no softmax is needed.

    def forward(self, X):
        for layer in self.layers:
            X = layer(X)
        return X
```

This is broken down in *Section 5: ResNet*. Furthermore, the following Appendix references to should allow more detailed understanding on ResNet architecture and model structure.

Reference *Appendix 9.7 for Images as Tensor*

Reference *Appendix 9.8 for Convolution Layer*

Reference *Appendix 9.9 for Pooling Operation*

Reference *Appendix 9.10 for Covariate Shift & Internal Covariate Shift*

Reference *Appendix 9.11 for Batch Normalization*

Reference *Appendix 9.12 for Linear Layer*

Reference *Appendix 9.13 for Activation Function (ReLU)*

6.7 Loss Function

```
# Set up criterion, optimizer, and trainer as before:
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight.to(device))
```

We are utilizing the positive weight as discussed above.

Since each image can have multiple labels, we use Binary Cross-Entropy with Logits Loss rather than Softmax Cross-Entropy.

In multi-label classification, where each image can have multiple labels (instead of exactly one class like in standard classification), we use Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss) rather than Softmax Cross-Entropy.

Softmax Cross-Entropy (For Single Label Classification)

- Used when each sample belongs to only one class.
- Applies Softmax to convert raw logits into a probability distribution.
- The loss function encourages the model to assign high probability to one correct class and low to others.

Binary Cross-Entropy (For Multi-Label Classification)

- Instead of treating the output as one probability distribution, it treats each class independently.
- Each class is treated as an independent binary classification problem.
- Loss is computed for every class, even those not present.
- Uses Sigmoid activation, which converts raw logits into independent probability scores for each class.
- Allows multiple classes to be predicted as "correct" for each sample.

For a given image x_i with M possible labels, the model outputs a raw score (logit) for each class $h_m(x_i)$. The sigmoid activation function is applied to convert each logit into a probability:

$$p_m(x_i) = \sigma(h_m(x_i)) = \frac{1}{1 + e^{-h_m(x_i)}}$$

where $\sigma(z)$ is the sigmoid function.

The Softmax Cross-Entropy loss for each class is:

$$l(y_i, p(x_i)) = - \sum_m^M (y_{i,m} \cdot \log\{p_m(x_i)\})$$

- $y_{i,m} \in \{0, 1\}$ is the one hot encoded true label.
- The loss only considers the one correct class in a single-label scenario.

The Binary Cross-Entropy (BCE) loss for each class is:

$$l(y_i, p(x_i)) = - \sum_m^M (y_{i,m} \cdot \log p_m(x_i) + (1 - y_{i,m}) \cdot \log\{(1 - p_m(x_i))\})$$

- $y_{i,m} \in \{0, 1\}$ is the true label for class m
- $p_m(x_i)$ is the predicted probability for class m
- The loss is computed independently for each class.

Please refer to Appendix

Given the class imbalance, we have also integrated weighted BCE:

$$l(y_i, p(x_i)) = - \sum_m^M (w \cdot y_{i,m} \cdot \log p_m(x_i) + (1 - y_{i,m}) \cdot \log\{(1 - p_m(x_i))\})$$

- $y_{i,m} \in 0, 1$ is the true label for class m
- $p_m(x_i)$ is the predicted probability for class m
- w is the weight which increases the penalty for misclassifying a positive example.
- The loss is computed independently for each class.

The weighting basically counts the positive and negative count for each class. We assume that N = total images and C is the set of all unique classes found in the "Finding Labels" column where $c \in C$.

$$\text{pos_counts}(c) = \sum_{i=1}^N I\{c \in y_i\}$$

where $I\{c \in y_i\}$ is an indicator function defined as:

$$I\{c \in y_i\} = \begin{cases} 1, & \text{if } c \in y_i \\ 0, & \text{if } c \notin y_i \end{cases}$$

And for the negative count for each class:

$$\text{neg_counts}(c) = N - \text{pos_counts}(c) = N \sum_{i=1}^N oI\{c \in y_i\}$$

Thus, the positive weight for a class c :

$$\text{pos_counts}(c) = \frac{N - \text{pos_counts}(c)}{\text{pos_counts}(c)}$$

Unique labels found: ['Atelectasis', 'Cardiomegaly', 'Consolidation', 'Edema', 'Effusion', 'Emphysema', 'Fibrosis', 'Hernia', 'Infiltration', 'Mass', 'No Finding', 'Nodule', 'Pleural_Thickening', 'Pneumonia', 'Pneumothorax']
 Positive weights: tensor([8.7806, 41.8432, 26.2743, 53.5924, 7.9896, 47.4660, 64.4198, 454.0179, 5.1238, 18.4809, 0.7897, 16.7136, 33.4454, 85.5229, 22.5608])

As we can see, given the class imbalance, 'Hernia' (which is the least common class from the dataset), has a higher weighting of 454.0179 compared to a very common class like 'No Finding' which has weighting 0.7897.

Why is BCE better than normal CE for the dataset?

- Since each image can have multiple diseases, it is a multi-label classification task.
- Softmax Cross-Entropy (CE) cannot be used, as it assumes that only one class is correct per image.
- Instead, we use Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss), which treats each class independently.

6.8 Optimizer (Adam)

```
optimizer = optim.Adam(my_resnet.parameters(), lr=0.0001)
```

Adam (Adaptive Moment Estimation) is an optimization algorithm used in deep learning to update the model's weights (parameters) based on the calculated gradients during training.

Adam

Not in assessment

- The best variant that essentially combines RMSProp with momentum
- Suggested default values:
 $\eta = 0.001$, $\alpha_1 = 0.9$, $\alpha_2 = 0.999$ and $\epsilon = 10^{-6}$.

Adam

- **Input:** $\eta > 0, \epsilon > 0 (10^{-6}), \beta_1, \beta_2 \in [0,1]$ and initial model θ

```

t = 1
while stopping criterion not met do
  Sample a mini-batch  $\{(x^1, y^1), \dots, (x^b, y^b)\}$ 
  Compute  $g = \frac{1}{b} \sum_{i=1}^b \nabla_{\theta} l(f(x^i, \theta), y^i)$ 
  Accumulate the gradient:  $s = \beta_1 s + (1 - \beta_1)g$ 
  Accumulate the square gradient:  $\gamma = \beta_2 \gamma + (1 - \beta_2)g \odot g$ 
  Correct s:  $\hat{s} = \frac{s}{1 - \beta_1^t}$  # t is the current iteration
  Correct r:  $\hat{r} = \frac{r}{1 - \beta_2^t}$ 
  Update  $\theta = \theta - \frac{\eta}{\sqrt{\epsilon + \hat{r}}} \odot \hat{s}$ 
  t = t + 1
end while

```

Note: $\odot$ means element-wise product

Figure 17: Adam Optimizer Monash University (Phung, 2024)

Prediction & Loss Function

a. Model Prediction

For a classification task, the model outputs probabilities for each of the M classes. The predicted class, denoted as $\hat{y}$, is chosen as the class with the highest probability:

$$\hat{y} = \operatorname{argmax}_{1 \leq m \leq M} p_m(x)$$

- $p_m(x)$: The probability that the input x belongs to class m .

b. Cross-Entropy Loss

To measure the discrepancy between the predicted probabilities and the true label, we use the cross-entropy loss. For a single training example, the loss is given by:

$$l(y, \hat{y}) = CE(1_y, p(x)) = -\log(p_{y_i}(x_i))$$

Breaking it down further, if the model's raw output (the logit) for class y is $h_y^L(x)$ then after applying the softmax function, the probability for class y becomes:

$$p_{y_i}(x_i) = \frac{e^{h_{y_i}^L(x_i)}}{\sum_{m=1}^M e^{h_{y_m}^L(x_i)}}$$

Thus, the loss can be written as:

$$l(y, \hat{y}) = -\log\left(\frac{e^{h_{y_i}^L(x_i)}}{\sum_{m=1}^M e^{h_{y_m}^L(x_i)}}\right)$$

Loss Over the Dataset

The loss over the entire training set (dataset, D , of size N) is the average cross-entropy loss:

$$L(D; \theta) = \frac{1}{N} \sum_{i=1}^N CE(1_{y_i}, p(x_i)) = -\frac{1}{N} \sum_{i=1}^N \log p_{y_i}(x_i)$$

- θ : The collection of all model parameters (weights and biases).
- x_i and y_i : The i th input and its true label.

The training objective is to find the parameters θ that minimize this loss:

$$\min_{\theta} L(D; \theta) = \min_{\theta} \left\{ \frac{1}{N} \sum_{i=1}^N l(f(x_i; \theta), y_i) \right\}$$

Here, $f(x_i; \theta)$ represents the model's prediction for x_i .

Mini-Batch Gradient Descent

Computing the gradient over the entire dataset is computationally expensive. Instead, we use a mini-batch of size b to estimate the gradient. For a mini-batch, the gradient estimate is:

$$\nabla_{\theta} \tilde{L}(D; \theta_t) := \frac{1}{b} \sum_{i=1}^b \nabla_{\theta} l(f(x_i; \theta), y_i)$$

i : Index of the example in the mini-batch.

θ_t : Model parameters at iteration t .

This mini-batch gradient is an unbiased estimator of the full gradient:

$$E[\nabla_{\theta} \tilde{L}(D; \theta_t)] = \nabla_{\theta} L(D; \theta_t)$$

A standard stochastic gradient descent (SGD) update would be:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \tilde{L}(D; \theta_t)$$

- $\nabla_{\theta} \tilde{L}(D; \theta_t)$ is the unbiased gradient estimation of the loss function with respect to the parameters.
- η is the learning rate, controlling the step size of each update.

Adam Optimizer Mechanics

Adam builds on SGD by incorporating two key ideas: momentum (first moment estimate) and adaptive learning rates (second moment estimate).

a. First Moment Estimate (Momentum)

Adam computes an exponentially decaying average of past gradients:

$$s_t = \beta_1 s_{t-1} + (1 - \beta_1) g_t$$

- g_t : The gradient at iteration t (i.e. $g_t = \frac{1}{b} \sum_{i=1}^b \nabla_{\theta} l(f(x_i; \theta), y_i)$)
- β_1 is the momentum coefficient (decay rate for first moment)

s_t keeps a running average of past gradients (this prevents large updates). It smooths the gradient updates. In other words, it helps the model accelerate updates in preferred directions by accumulating gradients in prior steps instead of jumping directions erratically. It helps accelerate convergence and reduce oscillations.

Since s_t starts from zero, it tends to be biased towards zero in early iterations. To correct this, we use:

$$\hat{s}_t = \frac{s_t}{1 - \beta_1^t}$$

- t is the current iteration. We are raising β to the t power.
- As t increases, β decreases. Why is this? At the beginning, β is large, and we do $1 - \beta$ which makes the denominator small, and thus makes $\hat{\gamma}_t$ large. This scales up the moment estimates in the early t iterations, correcting for the bias introduced by starting at zero. When t increases, β gets very small. At that point, the bias correction becomes negligible, and the moment estimates are essentially the true moving averages.

b. Second Moment Estimate (RMSProp)

Simultaneously, Adam maintains an exponentially decaying average of the squared gradients:

$$\gamma_t = \beta_2 \gamma_{t-1} + (1 - \beta_2) g_t \odot g_t$$

- β_2 is the RMSProp decay rate (default: 0.999)
- γ_t represents the variance of the gradient updates. It seen as an adaptive learning rate. In other words, it adjusts how much you move.
 - If a parameter's gradient is large, the learning rate is reduced for stability.
 - If a parameter's gradient is small, the learning rate is increased to allow faster convergence.
- This helps scale the updates based on past gradient magnitudes
- It adapts learning rates for each parameter based on how frequently that parameter changes.

To correct bias in early iterations, we use:

$$\hat{\gamma}_t = \frac{\gamma_t}{1 - \beta_2^t}$$

- t is the current iteration. We are raising β to the t power.
- As t increases, β decreases. Why is this? At the beginning, β is large, and we do $1 - \beta$ which makes the denominator small, and thus makes $\hat{\gamma}_t$ large. This scales up the moment estimates in the early t iterations, correcting for the bias introduced by starting at zero. When t increases, β gets very small. At that point, the bias correction becomes negligible, and the moment estimates are essentially the true moving averages.

Adam Parameter Update

Using the bias-corrected estimates, Adam updates the parameters as follows:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{\gamma}_t} + \epsilon} \odot \hat{s}_t$$

- ϵ : A small constant (e.g 10^{-8}), to prevent division by zero.
- The term $\frac{\eta}{\sqrt{\hat{\gamma}_t} + \epsilon}$ adjusts the learning rate for each parameter individually based on the historical gradient variance.
- $\hat{s}_t$ determines the direction of the update (smoothed gradient).

Why Adam Is Effective Compared to SGD

- Adam adjusts the learning rate for each parameter based on the magnitude of past gradients. This is especially useful when gradients vary widely in scale across parameters.
- The first moment estimate smooths the gradient updates, reducing the effect of noisy gradients in early training and helping the optimizer accelerate in consistent directions.

- The second moment estimate (RMSProp component) scales down the learning rate in regions with high gradient variance, which dampens oscillations and improves convergence.

7. Results & Evaluation

7.1 Confusion Matrix Theory

Before discussing the results and the evaluations, we must cover mathematically what these values mean.

		Predicted class	
		P	N
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

Sensitivity Recall P (yellow box around TP and FN)
 Specificity N (green box around FP and TN)
 Precision (red box around TP and FP)

Figure 18: Confusion Matrix (Harun, 2019)

Recall measures how many actual positive cases were correctly identified by the model.

$$Recall = \frac{TP}{TP + FN}$$

- High Recall means the model rarely misses positive cases.
- Low Recall means the model misses many actual positives (high FN).
- Important in medical diagnosis because missing a disease (FN) can be dangerous.

Precision measures how many of the predicted positive cases were actually correct.

$$Precision = \frac{TP}{TP + FP}$$

- High Precision means the model rarely makes mistakes when diagnosing someone with a sickness.
- Low Precision means the model has many false positives (FPs) (e.g. wrongly labels healthy people as sick).

Specificity measures how many of the actual negative cases were correctly identified.

$$Specificity = \frac{TN}{TN + FP}$$

- High Specificity means the model rarely makes mistakes when diagnosing someone as healthy (TN)
- Low Specificity means the model mistakenly classifies healthy people as sick (high FP).
- Important when false positives cause unnecessary treatments.

Metric	Formula	Question Answered
Recall (Sensitivity) (TPR)	$\frac{TP}{TP + FN}$	"Of all diseased patients, how many did we detect?"
Precision	$\frac{TP}{TP + FP}$	"Of all predicted diseased, how many were actually sick?"
Specificity	$\frac{TN}{TN + FP}$	"Of all healthy patients, how many did we correctly classify?"

7.2 ROC Curve Theory

The ROC Curve is a graph that shows how well a classification model can separate positive cases (diseased patients) from negative cases (healthy patients) at different thresholds.

- True Positive Rate (TPR) = Sensitivity = Recall (how many actual positives were correctly classified)
- False Positive Rate (FPR) (how many actual negatives were wrongly classified as positives)

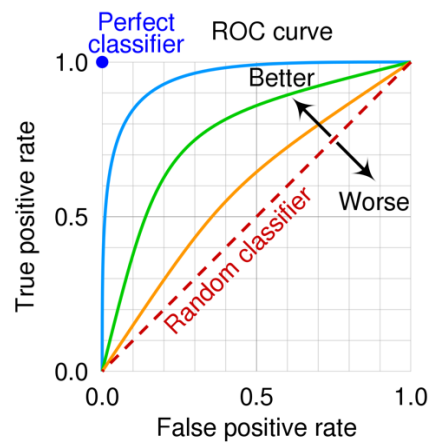


Figure 19: AUC Curve Example (Gomede, 2023)

Most classifiers output a probability score rather than a direct class label. To convert this probability into a classification, we set a threshold (e.g., 0.5). However, changing the threshold impacts false positives and true positives:

- A very low threshold (e.g., 0.1) will classify more instances as positive, increasing TPR, but also increasing FPR (more false alarms).
- A very high threshold (e.g., 0.9) will classify fewer instances as positive, reducing FPR, but also reducing TPR (missing real positives).

Why We Need an ROC Curve

A classifier (e.g., logistic regression) can output probabilities rather than hard predictions. To decide “positive” (e.g., obese) vs. “negative” (not obese), you choose a threshold (e.g., 0.5). But that threshold can vary. Different thresholds give different trade-offs between correctly identifying positives vs. accidentally flagging negatives as positive. We’d like a single plot that summarizes all possible thresholds in one place.

Construction of the ROC Curve

(1) We vary the threshold value (from 0 -> 1)

(2) Calculate the confusion matrix values for the model

(3) Calculate the TPR and FPR

(4) Plot TPR and FPR coordinate on the ROC curve

- Imagine sliding the threshold from very low (almost everything is called “positive”) to very high (almost everything is called “negative”).
- For each threshold value, you get a confusion matrix, which tells you:
 - True Positives (TP): Correctly predicted positives
 - False Positives (FP): Negatives incorrectly predicted as positives
 - True Negatives (TN): Correctly predicted negatives
 - False Negatives (FN): Positives incorrectly predicted as negatives

TPR: It measures what fraction of actual positives you caught.

FPR: It measures what fraction of actual negatives got misclassified as positive.

Interpreting the ROC Curve

The Diagonal (FPR = TPR line) is effectively random guessing.

A Good Curve bows toward the top-left corner (high TPR, low FPR).

You can see which threshold leads to the best balance for your application (depending on how much you care about false positives vs. true positives).

AUC (Area Under the Curve)

AUC is simply the area under the ROC curve, ranging from 0.0 to 1.0.

An AUC near 1.0 means the classifier is very good at ranking positives higher than negatives across all possible thresholds.

You can compare two models’ ROC curves easily by comparing their AUC: the model with the higher AUC is typically “better” overall.

Picking the “Best” Threshold

“Best” depends on your application. For instance:

- If missing a positive (e.g., failing to diagnose a serious disease) is very costly, you might choose a lower threshold to maximize TPR (catch more positives, even if that means more false alarms).

- If false alarms are very costly, you might want a higher threshold to minimize FPR.

The ROC curve visually shows that trade-off. You pick the point (threshold) on the curve that balances TPR vs. FPR appropriately for your needs.

A point where TPR = FPR means that the fraction of actual positives the model correctly identifies (true-positive rate) is numerically equal to the fraction of actual negatives it incorrectly flags as positive (false-positive rate). In other words, when these two quantities match, the model is correctly catching positives at the same rate that it's mislabeling negatives as positives.

7.3 Results

Below are results from the official paper deriving the results. The goal is to achieve similar results but utilizing our ResNet18 model.

Setting	Atelectasis	Cardiomegaly	Effusion	Infiltration	Mass	Nodule	Pneumonia	Pneumothorax
Initialization with different pre-trained models								
AlexNet	0.6458	0.6925	0.6642	0.6041	0.5644	0.6487	0.5493	0.7425
GoogLeNet	0.6307	0.7056	0.6876	0.6088	0.5363	0.5579	0.5990	0.7824
VGGNet-16	0.6281	0.7084	0.6502	0.5896	0.5103	0.6556	0.5100	0.7516
ResNet-50	0.7069	0.8141	0.7362	0.6128	0.5609	0.7164	0.6333	0.7891

Figure 20: AUCs of ROC curves for multi-label classification in different DCNN model setting (Wang, 2017)

Below are comparing the AUC curve from the official academic paper and my derived results:

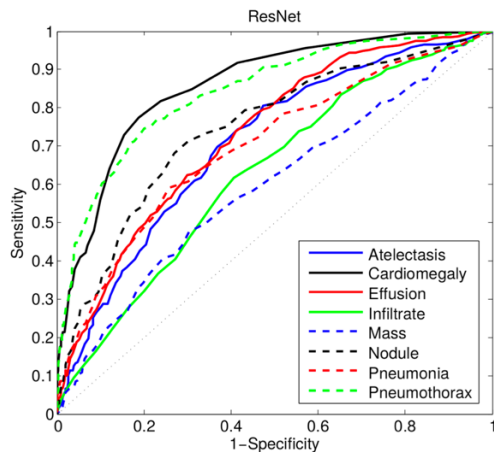


Figure 21: ROC Curve from Academic Paper

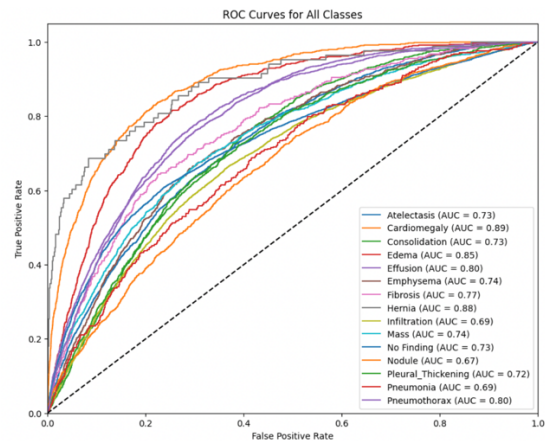


Figure 22: ROC Curve from Integrated Model

Figure 23: Comparing Disease AUC from Academic Paper with Integrated Model

Disease	AUC Academic Paper	AUC Integrated Model
Atelectasis	70.69%	73.25%
Cardiomegaly	81.41%	88.78%
Consolidation	-	72.72%

Edema	-	84.62%
Effusion	73.62%	80.42%
Emphysema	-	73.73%
Fibrosis	-	76.69%
Hernia	-	88.45%
Infiltration	61.28%	68.96%
Mass	56.09%	73.80%
No Finding	-	73.25%
Nodule	71.64%	66.50%
Pleural Thickening	-	71.86%
Pneumonia	63.33%	68.54%
Pneumothorax	78.91%	80.11%

The results show the integrated model generally outperforms the academic paper's model in most disease classifications, demonstrating improved classification capability.

Superior Performance: The integrated model achieves higher AUC scores across several key conditions, including Atelectasis (71.71% vs. 70.69%), Cardiomegaly (87.32% vs. 81.41%), Effusion (80.33% vs. 73.62%), Edema (82.74%), and Hernia (87.79%). This suggests the integrated model captures and differentiates between disease patterns, potentially benefiting from better feature extraction or fine-tuning techniques.

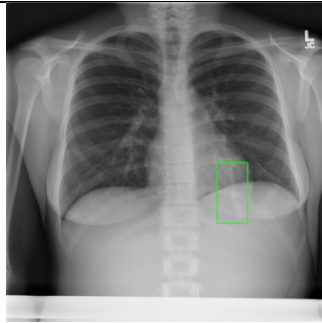
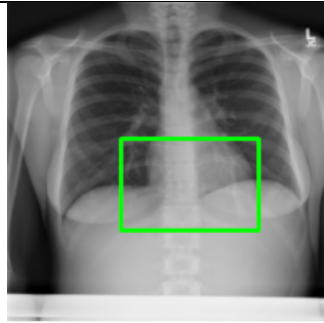
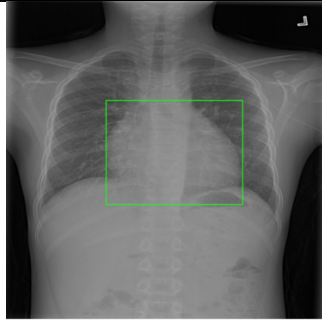
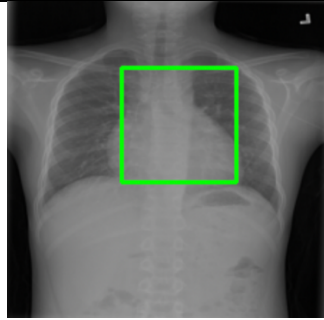
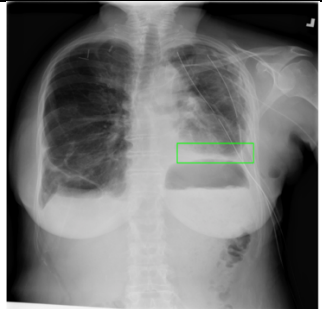
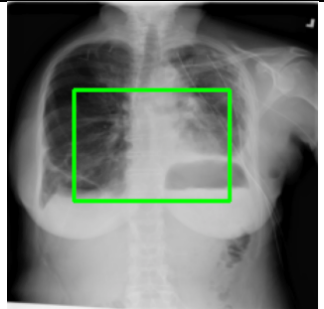
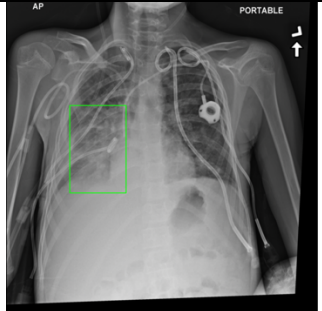
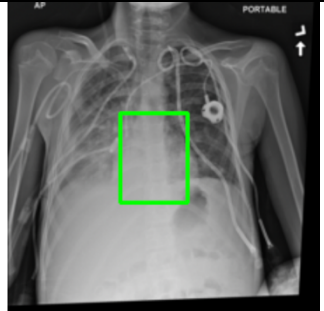
Significant Gains: Notably, Mass (71.80% vs. 56.09%) and Infiltration (67.39% vs. 61.28%) saw substantial improvements, indicating the integrated model's ability to handle more ambiguous or overlapping patterns in chest X-rays.

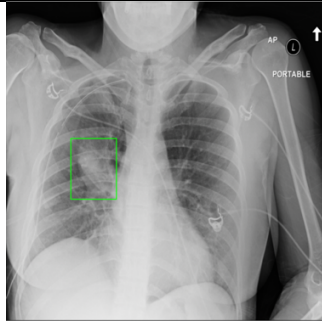
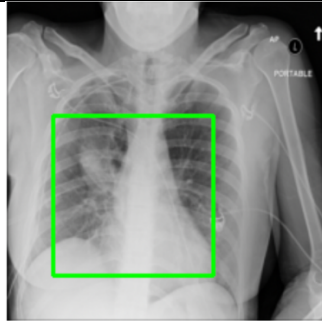
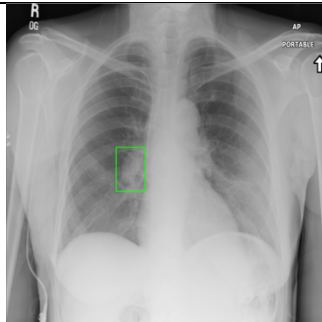
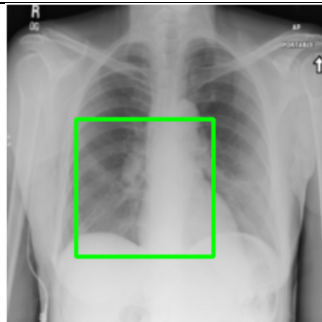
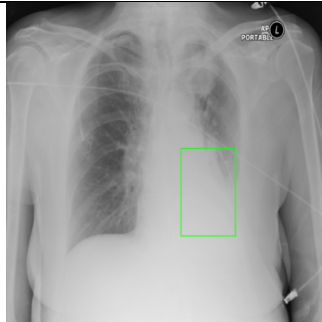
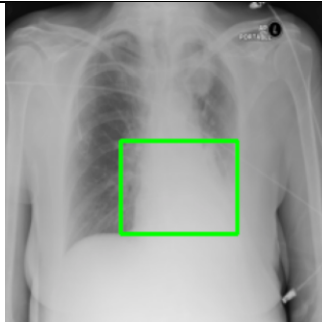
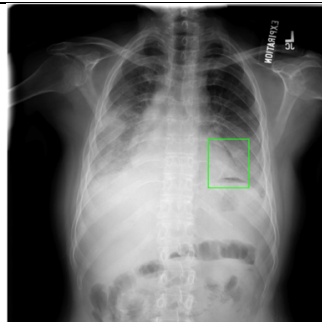
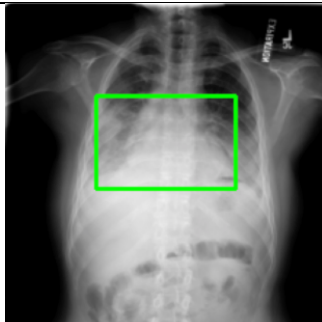
Comparable or Slight Decline: The Nodule (64.78% vs. 71.64%) and Pneumothorax (78.01% vs. 78.91%) results show a minor decrease in performance. This could suggest that the model may need further optimization for these specific disease categories, possibly requiring additional training data or targeted preprocessing.

We also integrated disease box detection using the feature maps (heat maps). Below are comparing a few example images with boxes from the 'BBox_List_2017.csv' file indicating the actual object detection and the model derivation through heat maps. The model derivation uses the feature maps to approximate spatial location of one particular thoracic disease class.

Reference *Appendix 9.14 Bounding Box Algorithm*

Figure 24: Bounding Box Locations from Actual Report vs Predicted

Disease	Image Name	Actual	Predicted
Atelectasis	00014198_000.png (index 8)		
Cardiomegaly	00028873_004.png (index 221)		
Consolidation	No record	No record	-
Edema	No record	No record	-
Effusion	00018366_029.png (index 332)		
Emphysema	No record	No record	-
Fibrosis	No record	No record	-
Hernia	No record	No record	-
Infiltration	00027441_002.png (index 582)		

Mass	00028330_003.png (index 646)		
Nodule	00019058_004.png (index 675)		
Pleural Thickening	No record	No record	-
Pneumonia	00013659_019.png (index 751)		
Pneumothorax			

8. Conclusion

Our approach incorporated strategies to address dataset imbalances, including down sampling the "No Finding" category and integrating a weighted Binary Cross-Entropy loss function to ensure better learning across all classes.

A key contribution of our work was the extraction of the last feature map from the ResNet model to enhance interpretability. By visualizing the regions influencing the model's predictions and enclosing them in bounding boxes, we provided additional justification for classification decisions, making the system more transparent.

The evaluation of our model using ROC curves and AUC scores demonstrated significant improvements over prior benchmarks. Our model outperformed the ResNet-50 results from the original academic paper in multiple disease categories, including Cardiomegaly, Effusion, and Infiltration. While performance was slightly lower in specific cases such as Nodule, these results highlight areas for further optimization, such as additional data augmentation, fine-tuning techniques, or alternative architectures.

Overall, this report shows the effectiveness of deep learning in medical imaging. Future work is to improve the model to match recent academic paper model accuracies and explore additional techniques such as attention mechanisms and or ensemble modeling.

9. Appendix

9.1 Derivative Proof

$$z_l = W_l \cdot a_{l-1} + b_l,$$

$$a_l = \sigma(z_l) = \sigma(W_l \cdot a_{l-1} + b_l)$$

To find the derivative of a_l , we need to compute the derivative of z_l with respect to a_{l-1} .

$$\frac{dz_l}{da_0} = z'_l = W_l$$

Therefore, we get:

$$\frac{da_l}{da_{l-1}} = \sigma'(z_l) \cdot W_l$$

9.2 Generalized Formula

To get the full generalized formula, we can utilize the Jacobian matrix:

□ $\bar{h} = xW + b$ and $h = \sigma(\bar{h})$

- $h = \sigma(xW + b)$
- σ is the activation function

□ $\frac{\partial h}{\partial x} = \frac{\partial h}{\partial \bar{h}} \times \frac{\partial \bar{h}}{\partial x} = \text{diag}(\sigma'(\bar{h})) W^T$

□ $\frac{\partial h}{\partial \bar{h}} = \begin{bmatrix} \frac{\partial h_1}{\partial \bar{h}_1} & \frac{\partial h_1}{\partial \bar{h}_2} & \frac{\partial h_1}{\partial \bar{h}_3} & \frac{\partial h_1}{\partial \bar{h}_4} \\ \frac{\partial h_2}{\partial \bar{h}_1} & \frac{\partial h_2}{\partial \bar{h}_2} & \frac{\partial h_2}{\partial \bar{h}_3} & \frac{\partial h_2}{\partial \bar{h}_4} \\ \frac{\partial h_3}{\partial \bar{h}_1} & \frac{\partial h_3}{\partial \bar{h}_2} & \frac{\partial h_3}{\partial \bar{h}_3} & \frac{\partial h_3}{\partial \bar{h}_4} \\ \frac{\partial h_4}{\partial \bar{h}_1} & \frac{\partial h_4}{\partial \bar{h}_2} & \frac{\partial h_4}{\partial \bar{h}_3} & \frac{\partial h_4}{\partial \bar{h}_4} \end{bmatrix} = \begin{bmatrix} \sigma'(\bar{h}_1) & 0 & 0 & 0 \\ 0 & \sigma'(\bar{h}_2) & 0 & 0 \\ 0 & 0 & \sigma'(\bar{h}_3) & 0 \\ 0 & 0 & 0 & \sigma'(\bar{h}_4) \end{bmatrix} = \text{diag}(\sigma'(\bar{h}))$

□ $\frac{\partial \bar{h}}{\partial x} = W^T$

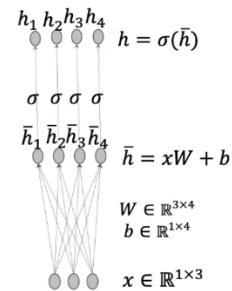


Figure 25: Lecture Slide on Chain Rule Back Propagation Monash University (Phung, 2024)

Remember $a_0 = x$ (as defined above).

$$\frac{da_l}{dx} = \frac{da_l}{dz_l} \cdot \frac{dz_l}{dx} = \text{diag}(\sigma'(z_l)) \cdot W_l$$

$$\frac{da}{dz} = \begin{bmatrix} \frac{da_1}{dz_1} \frac{da_1}{dz_2} \frac{da_1}{dz_3} \frac{da_1}{dz_4} \\ \frac{da_2}{dz_1} \frac{da_2}{dz_2} \frac{da_2}{dz_3} \frac{da_2}{dz_4} \\ \frac{da_3}{dz_1} \frac{da_3}{dz_2} \frac{da_3}{dz_3} \frac{da_3}{dz_4} \\ \frac{da_4}{dz_1} \frac{da_4}{dz_2} \frac{da_4}{dz_3} \frac{da_4}{dz_4} \end{bmatrix} = \begin{bmatrix} \sigma'(z_1) & 0 & 0 & 0 \\ 0 & \sigma'(z_2) & 0 & 0 \\ 0 & 0 & \sigma'(z_3) & 0 \\ 0 & 0 & 0 & \sigma'(z_4) \end{bmatrix} = \text{diag}(\sigma'(z))$$

$$\frac{dz_l}{dx} = \frac{dz_l}{da_0} = W_l$$

- The $\text{diag}()$ represents all the derivatives for each layer in the model.
- W represents all the layers in the model.

Therefore, the formula is:

$$J_l = \frac{da_l}{da_{l-1}} = \text{diag}(\sigma'(z_l)) \cdot W_l$$

9.3 What is a norm?

A norm is a way to measure the "size" or "magnitude" of a vector or matrix.

This norm ($\|W\|$) tells you the maximum factor by which the matrix W can stretch any vector x . In other words, if you multiply a vector by W , its length can increase by at most a factor of $\|W\|$.

9.4 What happens if W_l is < 1 ? (vanishing gradient)

Suppose $c = 0.25$ (using the sigmoid's maximum derivative) and let's say $W = 0.5$. Then:

$$c \cdot W = 0.25 \times 0.5 = 0.125.$$

The gradient scales as:

$$(0.125)^L.$$

Even for a small number of layers, this product becomes very small. For example, if $L = 4$:

$$(0.125)^4 = 0.00024414,$$

which is nearly zero.

9.5 What happens if W_l is > 1 ? (exploding gradient)

Now suppose $c = 0.25$ but the weights are much larger, say $W = 10$. Then:

$$c \cdot W = 0.25 \times 10 = 2.5.$$

The gradient scales as:

$$(2.5)^L.$$

For $L = 4$:

$$(2.5)^4 = 39.0625,$$

which is very large. With even more layers, this number grows exponentially, hence an exploding gradient.

9.6 Forward Propagation Example

Let's assume we have 3 layers (i.e., $L = 3$), so:

1. Layer 1 to Layer 2:

$$x_2 = g(x_1) = x_1 + F(x_1)$$

2. Layer 2 to Layer 3:

$$x_3 = g(x_2) = x_2 + F(x_2)$$

3. Generalizing for $L = 3$:

Substituting x_2 from step 1 into step 2:

$$x_3 = g(x_2) = (x_1 + F(x_1)) + F(x_2)$$

$$x_3 = g(x_2) = x_1 + F(x_1) + F(x_2)$$

In general, for any L :

$$x_L = g(x_{L-1}) = x_1 + \sum_{i=1}^{L-1} F(x_i)$$

9.7 Images as Tensor

Images as Tensor

```
# Load sample images
china = load_sample_image("china.jpg")[80:360, 70:390]
flower = load_sample_image("flower.jpg")[80:360, 130:450]
batch = np.array([china, flower], dtype=np.float32)
batch = np.transpose(batch, (0, 3, 1, 2))
print(batch.shape)
```

(2, 3, 280, 320)

2 Images
height
width
RGB channels

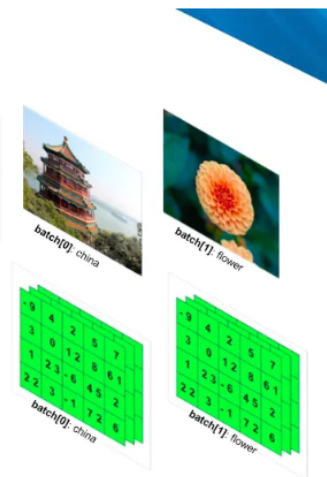


Figure 26: Image as Tensor (Phung, 2024)

9.8 Convolution Layer

The convolution operation is the core mechanism that extracts features from input data (most commonly images).

Local Receptive Fields (Filters/Kernels)

- A convolution uses small, learnable filters (sometimes called kernels) that slide over the input.
- These filters are much smaller than the original input image (for example, a 3×3 or 5×5 patch for a typical image).

- Each filter looks at a small region of the input at a time.

Strides and Padding

- Stride determines how far the filter moves each time it slides across the input (e.g., a stride of 1 moves one pixel at a time; a stride of 2 moves two pixels at a time).
- Padding (often zeros) can be added to control the spatial output size. This can help preserve the input's edge information.

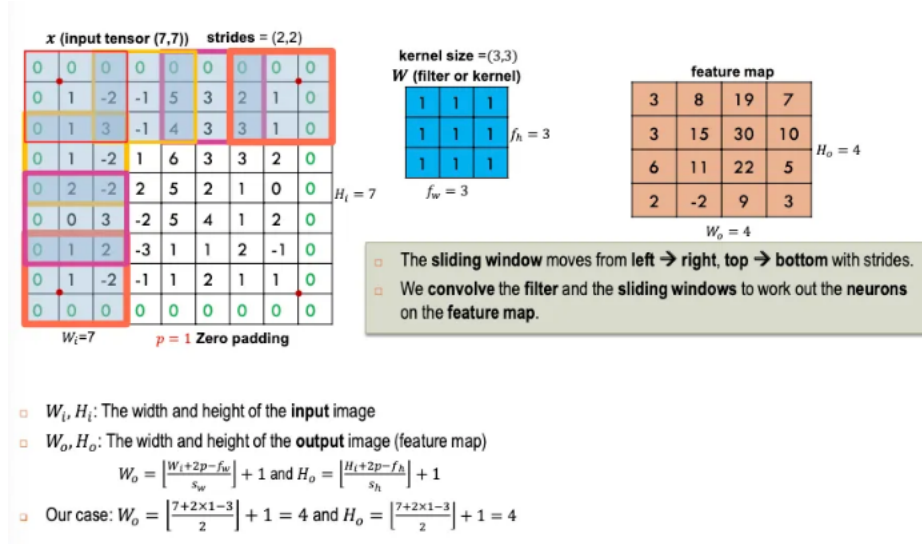


Figure 27: Convolution Layer (Phung, 2024)

$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s_w} \right\rfloor + 1$$

$$H_o = \left\lfloor \frac{H_i + 2p - f_h}{s_h} \right\rfloor + 1$$

- W_i, H_i : The width and height of the input image/tensor
- W_o, H_o : The width and height of the output image/tensor or feature map
- f_w and f_h is filter size (square)
- s_w and s_h is the stride
- p is the padding

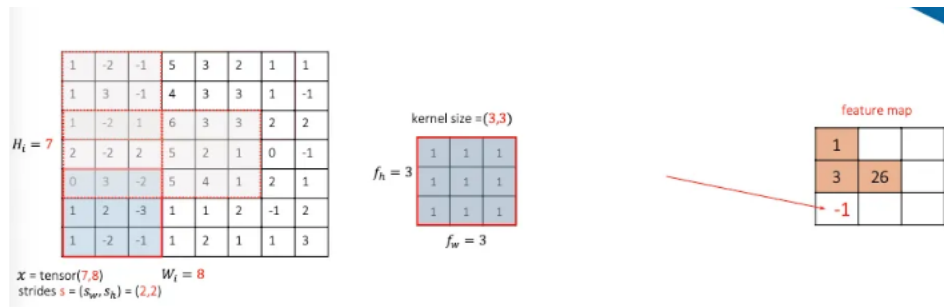


Figure 28: Convolution 2D without zero padding (Phung, 2024)

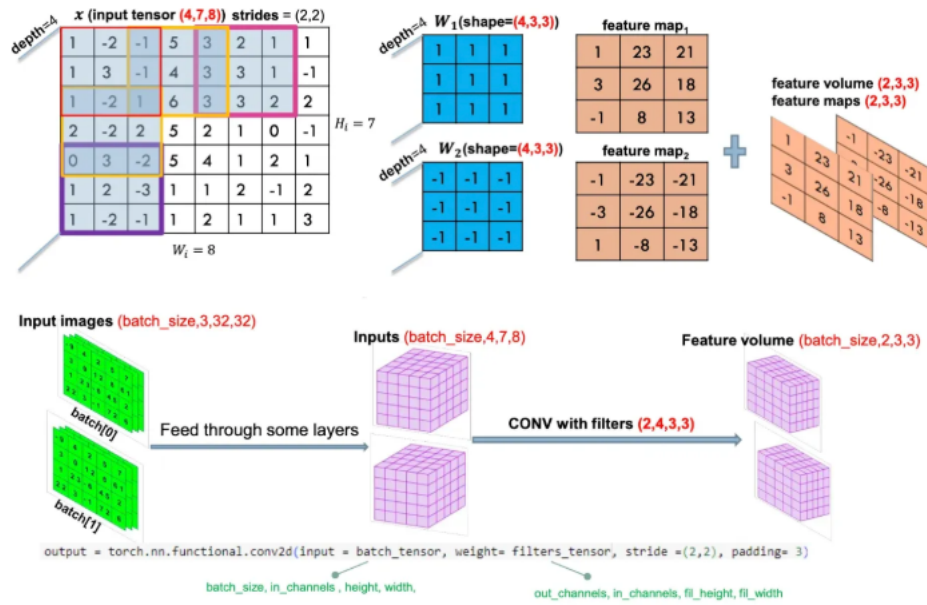


Figure 29: Feature Map Creation and Dimension Calculation (Phung, 2024)

X is the input tensor.

- Batch size of the images (dataset) of size B (batch_size)
- Filter size = (2, 4, 3, 3)
 - 2 filters
 - 4 depth (channels)
 - Size 3x3
- It has a depth (the number of channels, C) = 4
- It has a height (H) of 7
- It has a width (W) of 8
- There is a stride (s) = 2
- The kernel is of size $(3 \times 3) = 3$

Therefore,

$$W_o = \left\lceil \frac{8 + 2 \cdot 0 - 3}{2} \right\rceil + 1 = 3.5 \approx 3$$

$$H_o = \left\lceil \frac{7 + 2 \cdot 0 - 3}{2} \right\rceil + 1 = 3$$

Therefore, we have output (batch_size, 2, 3, 3)

- The channels/depth output size changes to the number of filters. Why?
 - Because the individual filter goes through each image's: depth x height x width. That's one filter output.
 - Therefore, given that there are two filters (with different value structure), it will output two image feature maps.

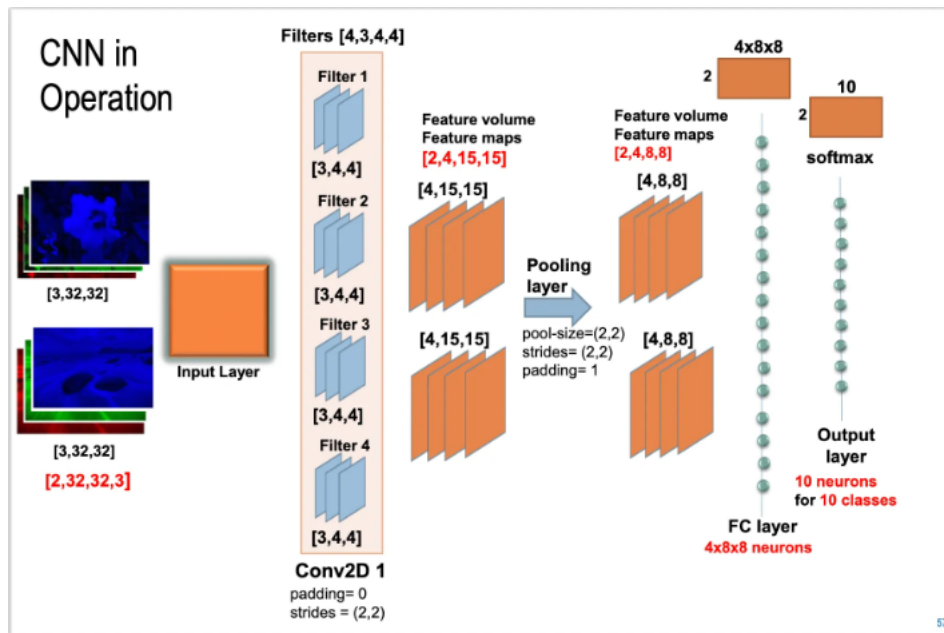


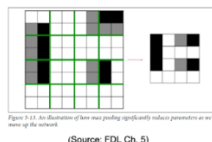
Figure 30: CNN in Operation (Phung, 2024)

The last 3D tensor [4,4,8] → 1 flat layer with 4x4x8 = 128 neurons (for two images, two 'batches')

9.9 Pooling Operation

Pooling operation

- Makes the representations smaller and more manageable
- Subsample the image
- Operates over each feature map independently



Pooling operation

- Makes the representations smaller and more manageable
- Operates over each channel independently

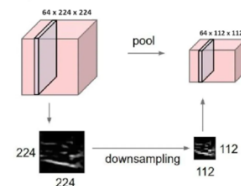


Figure 31: Pooling Operation (Phung, 2024)

Max/average pooling

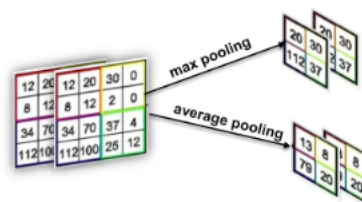
- Max/average-pooling is **locally invariant** and applied **independently to each feature map**
- In the SOTA CNNs (ResNet, GoogleNet, VGG), we employ the pooling layer with

- Kernel/filter size = (2,2), stride = (2,2), padding = 0
- Therefore, the output tensor has the size

$$W_o = \left\lfloor \frac{224+0-2}{2} \right\rfloor + 1 = 112$$

$$H_o = \left\lfloor \frac{224+0-2}{2} \right\rfloor + 1 = 112$$

- With this setting, we **down-sample** the input size by 2



(Source: medium.com)

Figure 32: Max Pooling and Average Pooling (Phung, 2024)

What Does “Pooling Operation” Mean?

- Pooling is a down-sampling (or sub-sampling) method used in convolutional neural networks (CNNs).
- It reduces the spatial dimensions (height and width) of the feature maps, while *keeping* the same number of channels.
- Common types:
 - Max Pooling: Take the maximum value in each local region.
 - Average Pooling: Take the average value in each local region.

Why Do We Pool?

1. Makes Representations Smaller & More Manageable

- By reducing spatial dimensions (e.g., from 224×224 to 112×112), the number of parameters and computations further down the network shrinks dramatically.
- This helps the network run faster and use less memory.

2. Helps Avoid Overfitting

- Because we effectively compress or subsample the feature maps, the model is forced to learn coarser, more “summarized” features instead of remembering exact pixel positions.

3. Builds Invariance to Small Translations

- For example, in max pooling, if a feature (like an edge) shifts by one pixel, it can still produce a similar pooled output. This gives the network some basic robustness to small shifts.

4. Operates Per Feature Map (Channel)

- Pooling runs over each channel independently. There’s no mixing between the channels during pooling (that typically happens in convolutional layers if at all).
- So if you have 64 channels, you’ll do 64 independent pooling operations, one for each channel.

How Does Pooling Actually Work?

$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s_w} \right\rfloor + 1$$

$$H_o = \left\lfloor \frac{H_i + 2p - f_h}{s_h} \right\rfloor + 1$$

- f_w and f_h is filter size (square)
- s_w and s_h is the stride
- p is the padding

1. Sliding Window / Kernel

- You choose a kernel size, e.g. 2×2, meaning you look at 2×2 cells at a time in the feature map.
- You choose a stride, often 2, so you move that 2×2 window 2 pixels at a time in both the height and width directions.
- No overlap happens if kernel=2×2 and stride=2 (that’s typical).

2. Apply the Pooling Function

- In max pooling, you pick the maximum value in each 2×2 patch.
- In average pooling, you average the values in the 2×2 patch.
- The result is a single number (max or average) that goes into the pooled output.

3. New Dimensions

As in Figure 31, if the original width W is 224, and you use a kernel of 2×2 and stride=2, you end up with roughly:

$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s_w} \right\rfloor + 1$$

- f_w is filter size (square) = 2
- s_w is the stride = 2
- p is the padding = 0

$$W_o = \left\lfloor \frac{224 + 2 \cdot 0 - 2}{2} \right\rfloor + 1 = \left\lfloor \frac{224 - 2}{2} \right\rfloor + 1 = \left\lfloor \frac{222}{2} \right\rfloor + 1 = 112$$

- The same applies for height.
- The number of channels stays the same, because you pool each feature map independently.

9.10 Covariate Shift & Internal Covariate Shift

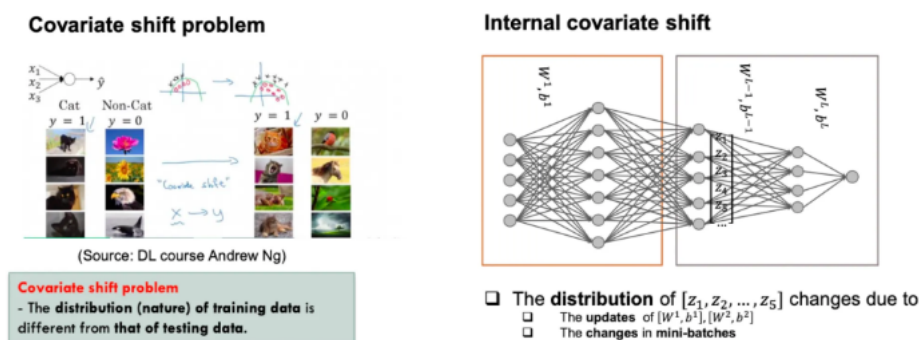


Figure 33: Covariate Shift vs Internal Covariate Shift (Phung, 2024)

Covariate Shift

- Occurs when the training data distribution is different from the test (inference) data distribution.
 - For example, you train a “cat vs. non-cat” model primarily on black cats and yellow flowers, but then at test time you see white cats and red flowers. The input distribution has shifted, which can degrade performance.

Internal Covariate Shift

- Within a neural network, each layer passes an output distribution to the next layer.
- As the network trains, each layer’s parameters change.
- So the distribution of these outputs can shift (sometimes drastically) between training iterations.
- This is called *internal* covariate shift because it’s happening *inside* the network layers.
- A shifting distribution makes training more difficult because:

- The next layer constantly needs to “relearn” how to adjust to new input ranges, which can slow convergence or lead to instability.

9.11 Batch Normalization

Batch Normalization

1. Cope with internal covariate shift
2. Reduce gradient vanishing/exploding
3. Reduce overfitting
4. Make training more stable
5. Converge faster

1. Allow us to train with bigger learning rate

- Let $z = W^k h^k + b^k$ be the mini-batch before activation. We compute the normalized $\hat{z}$ as
 - $\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ where ϵ is a small value such as $1e^{-7}$
 - $\mu_B = \frac{1}{b} \sum_{i=1}^b z_i$ is the empirical mean
 - $\sigma_B^2 = \frac{1}{b} \sum_{i=1}^b (z_i - \mu_B)^2$ is the empirical variance
- We scale the normalized $\hat{z}$
 - $z_{BN} = \gamma \hat{z} + \beta$ where $\gamma, \beta > 0$ are two learnable parameters (i.e., scale and shift parameters)
- We then apply the activation to obtain the next layer value
 - $h^{k+1} = \sigma(z_{BN})$

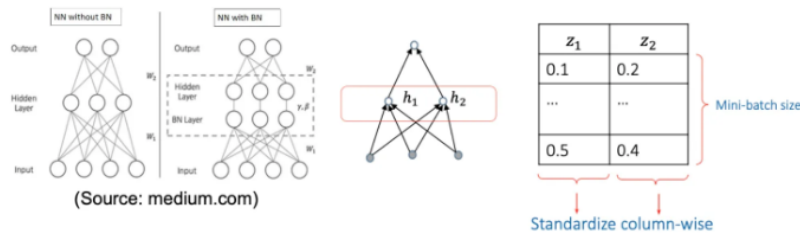


Figure 34: Batch Normalisation (Phung, 2024)

Goal: BN reduces internal covariate shift by stabilizing the distribution of layer inputs over the course of training.

Gamma and Beta (the learnable parameters) for the Batch Normalisation is of size C (the size of the channels).

So what is the process?

You feed the layer an input (which is h_k from the previous layer).

The layer does its linear (or convolutional) computation: multiply by weights W_k , add bias b_k to get z_k .

$$z^k = W^k h^k + b^k$$

BatchNorm normalizes this z_k across the batch (computing mean/variance and then applying γ, β).

- Reference μ from the video above
- Reference σ from the video above

$$\widehat{z^k} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$z_{BN}^k = \widehat{z^k} \cdot \gamma +$$

- You apply an activation function to that batch-normalized output to get h_{k+1} .

$$h^{k+1} = \sigma(z_{BN}^k)$$

- h_{k+1} is then passed forward into the next layer ($k + 1$).

How does batch normalization help with Internal Covariate Shift?

- Normalizes the Distribution for Each Channel
 - By centering (subtracting mean) and rescaling (dividing by standard deviation), BatchNorm keeps each channel's distribution close to mean 0 and variance 1 over the mini-batch.
 - This makes the distribution that the next layer sees much more stable through training, even if earlier layers' weights change.
- Applies a Learnable Shift & Scale (γ and β)
 - After normalization, BatchNorm includes trainable parameters (γ and β for each channel).
 - These let the network "undo" or adjust the normalization if it's beneficial for learning (e.g., if a certain channel actually learns best with mean=2, variance=4).
- Stabilizes Activations & Gradients
 - Because the inputs to each layer stay roughly on a consistent scale, you avoid issues like exploding or vanishing gradients.
 - Gradients flow more reliably, which typically leads to faster convergence.

9.12 Linear Layer

Often used as the last layer in a network to produce outputs after feature extraction has been performed by other layers (e.g., after convolutional layers).

It essentially multiplies the input X by a weight matrix W and adds a bias b .

$$\text{output} = X \times W^T + b$$

- Weight matrix and bias matrix are learnable parameters during back propagation

Typically, you specify `in_features` (the size of each input sample) and `out_features` (the size of each output sample).

- For example, in a ResNet model, `out_features` would be the number of classes (e.g., 15).
- If you have input of shape $(\text{batch_size}, N)$, then `in_features = N`.

```
(1, 64,112,112) ← nn.LazyConv2d(64, kernel_size=7, stride=2, padding=3),
(1, 64,112,112) ← nn.LazyBatchNorm2d(),
(1, 64,112,112) ← nn.ReLU(),
(1, 64,56,56) ← nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
(1, 64,56,56) ← ResnetBlock(64, 2, first_block=True),
(1, 128,28,28) ← ResnetBlock(128, 2),
(1, 256,14,14) ← ResnetBlock(256, 2),
(1, 256,1,1) ← nn.AdaptiveAvgPool2d((1, 1)),
(1,256) ← nn.Flatten(1),
(1,10) ← nn.LazyLinear(10),
# nn.Softmax(dim=-1)
])
```

Figure 35: Dimensions of input through ResNet model (Phung, 2024)

LazyLinear

- Performs the same linear operation, but you don't specify `in_features` up front.
- When the layer sees its first batch of data, it infers `in_features` from the input's shape, then initializes the weights and bias.

- You still provide *out_features* explicitly.

9.13 Activation Function (ReLU)

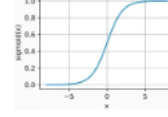
Modern Activation Functions



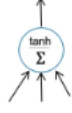
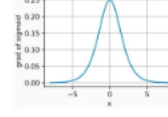
□ The **sigmoid** function

- $\sigma(z) = \frac{1}{1+e^{-z}}$
- Logistic S-shaped, continuous, and differentiable
- Output range: 0 to 1

$$\sigma(z) = s(z) = \frac{1}{1 + \exp\{-z\}}$$



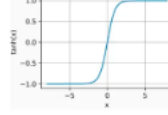
$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$



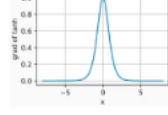
□ The **hyperbolic tangent** function

- $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- Logistic S-shaped, continuous, and differentiable
- Output range: -1 to 1, hence help speed up convergence.

$$\sigma(z) = \tanh(z) = \frac{\exp\{z\} - \exp\{-z\}}{\exp\{z\} + \exp\{-z\}}$$



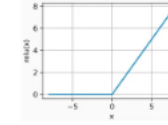
$$\sigma'(z) = 1 - \sigma^2(z)$$



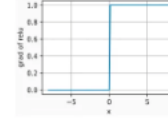
□ The **rectified linear function** (ReLU)

- $\text{ReLU}(z) = \max(0, z)$
- Continuous, but not differentiable at 0
- Hence, can make GD bounce around
- Fast to compute, works very well in practice!
- No cap on output value, hence help with gradient vanishing problem.

$$\sigma(z) = \text{ReLU}(z) = \max\{0, z\}$$



$$\sigma'(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



30

Figure 36: Modern Activation Functions (Phung, 2024)

ReLU sets all negative values to 0 and leaves positive values unchanged.

Therefore, there is no change in the dimensionality of the input. Reference, Figure 36 above.

9.14 Bounding Box Algorithm

Feature Map Extraction from Convolutional Layer

Given an input image X with dimensions $(1, C, H, W)$, the model extracts feature maps from a target convolutional layer.

Let F be the feature map tensor output at layer l having dimensions:

$$F \in \mathbb{R}^{1 \times N \times H' \times W'}$$

Where:

- N is the number of feature channels (filters)
- H' and W' are the spatial dimensions of the feature map (usually smaller than H, W due to pooling).

Let's say for a hypothetical example, we have 256 channels and the height (H' , after pooling) is 8 and the width (W' , after pooling) is 8. Thus, we have:

$$F \in \mathbb{R}^{1 \times 256 \times 8 \times 8}$$

The code selects the channel (out of the N) with the finding the maximum in the image. For simplicity, let's assume we select the last channel so that:

$$h_{\max}(i, j) = \max_{c=1, \dots, N} F(0, c, i, j) = F_{\max}(i, j)$$

This produces a 2D heatmap $h \in \mathbb{R}^{H \times W}$ that highlights the strongest activation at each spatial location.

For our hypothetical example, suppose this chosen feature map F_{max} is:

$$h = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.2 & 0.1 & 0.0 & 0.0 & 0.0 \\ 0.1 & 0.4 & 0.8 & 0.7 & 0.2 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.3 & 0.9 & 1.0 & 0.4 & 0.1 & 0.0 & 0.0 \\ 0.0 & 0.2 & 0.7 & 0.8 & 0.3 & 0.1 & 0.0 & 0.0 \\ 0.0 & 0.1 & 0.4 & 0.3 & 0.1 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.1 & 0.1 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix}$$

This is what your function `compute_heatmap(feature_maps)` returns (after selecting the channel).

Normalization

Going forward we reference the feature map that we chose containing the maximum value as $F_{max}(i, j)$.

The raw heatmap may have arbitrary values. To bring it into a common scale, you normalize it to the range $[0, 1]$:

$$h(i, j) = \frac{F_{max}(i, j) - \min(F_{max})}{\max(F_{max}) - \min(F_{max})}$$

In our example,

$$\min(F_{max}) = 0$$

$$\max(F_{max}) = 1.0$$

So, the normalized heatmap is now:

$$h(i, j) = F_{last}(i, j)$$

Thus, the normalized heatmap h is:

$$h = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.2 & 0.1 & 0.0 & 0.0 & 0.0 \\ 0.1 & 0.4 & 0.8 & 0.7 & 0.2 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.3 & 0.9 & 1.0 & 0.4 & 0.1 & 0.0 & 0.0 \\ 0.0 & 0.2 & 0.7 & 0.8 & 0.3 & 0.1 & 0.0 & 0.0 \\ 0.0 & 0.1 & 0.4 & 0.3 & 0.1 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.1 & 0.1 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix}$$

Scaling to [0,255] and Thresholding

The normalized heatmap is then scaled to the 0–255 range:

$$h_{scaled}(i, j) = 255 \cdot h(i, j)$$

For instance:

- $0.1 \rightarrow 25$
- $0.2 \rightarrow 51$
- $0.3 \rightarrow 76$
- $0.8 \rightarrow 204$
- $1.0 \rightarrow 255$

Thus, the scaled heatmap H becomes approximately:

$$H = \begin{bmatrix} 25 & 51 & 76 & 51 & 25 & 0 & 0 & 0 \\ 25 & 102 & 204 & 179 & 51 & 0 & 0 & 0 \\ 0 & 76 & 229 & 255 & 102 & 25 & 0 & 0 \\ 0 & 51 & 179 & 204 & 76 & 25 & 0 & 0 \\ 0 & 25 & 102 & 76 & 25 & 0 & 0 & 0 \\ 0 & 0 & 25 & 25 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Now, the process applies two thresholds: $T_1 = 60$ and $T_2 = 180$. For each threshold T , we create a binary map B_T defined as:

$$B_T(i,j) = \begin{cases} 255, & \text{if } H(i,j) \geq T, \\ 0 & \text{otherwise.} \end{cases}$$

For $T_1 = 60$:

- Values < 60 become 0, and values ≥ 60 become 255.
- For instance, in our matrix, 25 and 51 become 0; 76, 102, 204, 229, 255 become 255.

For $T_2 = 180$:

- Values < 180 become 0, and values ≥ 180 become 255.
- For instance, in our matrix, 25 and 51 become 0; 76, 102, 204, 229, 255 become 255.

Then the two binary maps are combined using a bitwise OR, so that if a pixel is above either threshold, it is marked as activated:

$$B(i,j) = B_{60}(i,j) \text{ OR } B_{180}(i,j)$$

This combined binary map will highlight regions where the network's activation is above 60 (or 180) on the 0–255 scale.

Contour Detection and Bounding Box Calculation

Lets say we derive a 8x8 binary map B that looks like this:

$$B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 255 & 255 & 0 & 0 & 0 & 0 & 0 \\ 0 & 255 & 255 & 0 & 0 & 255 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 255 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 255 & 255 & 255 & 0 & 0 \\ 0 & 0 & 0 & 255 & 255 & 255 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Here, the value 255 represents the “active” or “hot” region (where the model's activation is high), and 0 represents the background.

Contours represent the boundaries of objects in an image. They are essentially a list of points that form a closed curve around an object.

The contour detection algorithm scans this binary image and looks for the boundaries of the white regions. Each contour is simply a list of points (x,y) that follow the edge of a connected region.

When you call OpenCV's

```
cv2.findContours(B, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
```

it returns a list of contours.

For each contour C_k , you can compute a bounding rectangle that minimally encloses the contour. This is done with a function like OpenCV's which returns:

$$cv2.boundingRect(C_k)$$

where:

- (x_k, y_k) is the top-left corner of the rectangle
- w_k is the width
- h_k the height

Union of Bounding Boxes

If there are multiple contours (multiple enclosed boundary coordinates), rather than taking each individual box, you can compute a single bounding box that encloses all of them. This is done by computing:

$$x_{min} = \min_k x_k$$

$$y_{min} = \min_k y_k$$

$$x_{max} = \max_k (x_k + w_k)$$

$$y_{max} = \max_k (y_k + h_k)$$

The union bounding box in heatmap coordinates is then:

$$BBox_{heatmap} = (x_{min}, y_{min}, x_{max} - x_{min}, y_{max} - y_{min})$$

Why do $x_{max} - x_{min}$ and $y_{max} - y_{min}$?

- For $BBox_{heatmap}$ it requires the actual height and weight in units (not the bottom left and bottom right coordinates)

Scaling the Bounding Box to the Target Image

Assume the heatmap has dimensions $H \times W$ (in our example, 8×8), and the target image (the input image to the model) is resized to 224×224 .

The scale factors are:

$$scale_x = \frac{244}{W} = \frac{244}{8} = 28$$

$$scale_y = \frac{244}{H} = \frac{244}{8} = 28$$

Thus, if the union bounding box in heatmap coordinates is:

$$BBox_{heatmap} = (x_{min}, y_{min}, w, h) = (1, 1, 3, 3)$$

then the bounding box in the target image space is computed as:

$$BBox_{scaled} = (x_{min} \times 28, y_{min} \times 28, 3 \times 28, 3 \times 28) = (28, 28, 84, 84)$$

Drawing the Bounding Box

Finally, you overlay the bounding box (28,28,84,84) on the original image (which has been resized to 224x224). In OpenCV, you use:

```
cv2.rectangle(image, (x, y), (x + w, y + h), color, thickness)
```

to draw a rectangle with the scaled coordinates.

9.15 Model Underfitting vs Overfitting

```
Epoch 1/10
1083/1083 - loss: 1.1389 - accuracy: 0.6993 - val_loss: 1.1128 - val_accuracy: 0.7030
Epoch 2/10
1083/1083 - loss: 1.0839 - accuracy: 0.7034 - val_loss: 1.1472 - val_accuracy: 0.7694
Epoch 3/10
1083/1083 - loss: 1.0433 - accuracy: 0.7112 - val_loss: 1.0440 - val_accuracy: 0.5958
Epoch 4/10
1083/1083 - loss: 1.0085 - accuracy: 0.7245 - val_loss: 1.0112 - val_accuracy: 0.7247
Epoch 5/10
1083/1083 - loss: 0.9912 - accuracy: 0.7314 - val_loss: 1.0041 - val_accuracy: 0.7090
Epoch 6/10
1083/1083 - loss: 0.9735 - accuracy: 0.7386 - val_loss: 0.9941 - val_accuracy: 0.7969
Epoch 7/10
1083/1083 - loss: 0.9594 - accuracy: 0.7430 - val_loss: 0.9806 - val_accuracy: 0.6854
Epoch 8/10
1083/1083 - loss: 0.9475 - accuracy: 0.7458 - val_loss: 0.9678 - val_accuracy: 0.6936
Epoch 9/10
1083/1083 - loss: 0.9364 - accuracy: 0.7484 - val_loss: 0.9486 - val_accuracy: 0.7447
Epoch 10/10
1083/1083 - loss: 0.9273 - accuracy: 0.7547 - val_loss: 0.9419 - val_accuracy: 0.7741
```

Figure 37: Training Verbose Output

Overfitting

- The model performs extremely well on the training set (low training loss, high training accuracy) but fails to generalize to unseen data (high validation loss, low validation accuracy).
- As training continues, the training loss keeps decreasing while the validation loss begins to rise or the validation accuracy plateaus or decreases. This indicates the model is memorizing the training data rather than learning generalizable features.

Underfitting

- The model does not perform well on both the training set and the validation set. This means it hasn't learned the underlying patterns sufficiently, often due to insufficient training or a model that's too simple for the task.
- Both training and validation losses remain relatively high, and accuracy remains low. Even if you train for more epochs, the model fails to improve significantly.

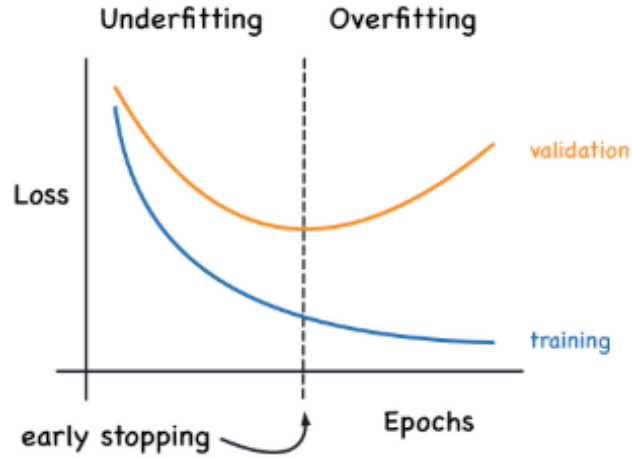


Figure 38: Underfitting vs Overfitting (Holbrook, 2024)

From Figure 36, we can see that:

Training Loss and Accuracy

- Training loss is steadily decreasing from 1.1389 to 0.9233.
- Training accuracy is steadily increasing from ~0.6993 to ~0.7547.

Validation Loss and Accuracy:

- Validation loss also trends downward overall (1.1128 → 0.9419), though it fluctuates slightly.
- Validation accuracy starts around 0.7930, hovers in the 0.79–0.80 range, dips at epoch 6 (0.6854), then recovers.

Thus, the fact that both training and validation losses are decreasing suggests that the model is still learning and is not heavily overfitting.

The validation accuracy remains fairly high and stable (around 0.79–0.80 early on), with one notable dip at epoch 6, but it recovers afterward.

9.16 Hyperparameters Used

The following hyperparameters were used in the model to derive the prediction results:

- Batch Size: 64
- Epochs: 15
- Learning Rate: 0.0001
- Optimizer: Adam
- Loss: Weighted BCE
- Image Resolution: Scaled from 1024x1024 to 256x256

9.17 BCE Loss Example

Assume we have a single image x_i , with 3 classes (this process generalizes to 15 classes). Let:

Ground truth vector:

$$y(x_i) = [1, 0, 1]$$

Model raw logits:

$$h(x_i) = [2.0, -1.0, 0.5]$$

Step 1: Compute the probabilities

Apply the sigmoid for each class:

For class 1:

$$p_1(x_i) = \sigma(2.0) = \frac{1}{1 + e^{-2.0}} \approx \frac{1}{1 + 0.1353} \approx 0.88$$

For class 2:

$$p_2(x_i) = \sigma(-1.0) = \frac{1}{1 + e^{1.0}} \approx \frac{1}{1 + 2.718} \approx 0.27$$

For class 3:

$$p_3(x_i) = \sigma(0.5) = \frac{1}{1 + e^{-0.5}} \approx \frac{1}{1 + 0.6065} \approx 0.62$$

Thus, $p(x_i) \approx [0.88, 0.27, 0.62]$

Step 2: Compute the BCE Loss for Each Class

For each class, use the BCE formula:

For class 1 ($y_{i,1} = 1$):

$$l_{i,1} = -[1 \cdot \log(0.88) + 0 \cdot \log(1 - 0.88)] \approx -\log(0.88) \approx 0.13$$

For class 2 ($y_{i,2} = 0$):

$$l_{i,2} = -[0 \cdot \log(0.27) + 1 \cdot \log(1 - 0.27)] \approx -\log(0.73) \approx 0.314$$

For class 3 ($y_{i,3} = 1$):

$$l_{i,3} = -[1 \cdot \log(0.62) + 0 \cdot \log(1 - 0.62)] \approx -\log(0.62) \approx 0.478$$

Step 3: Total Loss

The total loss for this image is the sum of the losses for each class:

$$l(x_i) = l_{i,1} + l_{i,2} + l_{i,3} \approx 0.13 + 0.314 + 0.478 \approx 0.922$$

For the medical example, there are 15 classes...

$$l(x_i) = - \sum_{m=1}^{15} [y_{i,m} \cdot \log(p_m(x_i)) + (1 - y_{i,m}) \cdot \log(1 - p_m(x_i))]$$

If we were using weighted BCE, then for each positive class the loss term would be multiplied by w .

For example, if the weight for class 1 is w_1 , then the loss for class 1 would be:

$$l_{i,1} = -[w_1 \cdot 1 \cdot \log(0.88) + 0 \cdot \log(1 - 0.88)] \approx -[w_1 \cdot \log(0.88)]$$

And similarly for the other classes.

10. Glossary

Chest X-ray: A radiographic imaging technique used to visualize the structures of the chest, including the lungs, heart, ribs, and diaphragm. It helps in diagnosing various thoracic diseases and conditions.

Thoracic A broad term referring to medical conditions that affect the organs within the thoracic (chest) cavity, including the lungs, heart, and pleura.

Atelectasis: Partial collapse of the lung; appears as a white, collapsed area with loss of lung volume.

Cardiomegaly: Enlarged heart; the heart appears larger than half the width of the chest.

Effusion: Fluid in the pleural space; appears as a white, hazy area at the bottom of the lung.

Infiltration: White spots on the lung indicating inflammation or infection.

Mass: A small, round spot in the lung, typically less than 3 cm in diameter; appears as a round, white area.

Nodule: A larger abnormal growth in the lung, exceeding 3 cm in size; appears as a round, white area.

Pneumonia: Infection causing lung inflammation; appears as patchy or widespread white areas in the lung.

Pneumothorax: Air in the pleural space; appears as a dark area with a sharp lung edge and no lung markings.

Consolidation: Dense lung tissue filled with fluid or pus; appears as a solid white area.

Edema: Fluid buildup in the lungs due to heart failure; appears as hazy, white areas in both lungs.

Emphysema: Damaged alveoli causing air trapping; appears as hyperinflated, darker lungs with less vascular markings.

Fibrosis: Thickening and scarring of lung tissue; appears as white, streaky lines.

Pleural Thickening: Thickened pleural lining; appears as a white band along the lung edges.

Hernia: Organs pushing through the diaphragm into the chest; appears as an abnormal soft tissue density.

Data Leakage: In data modelling, data leakage occurs when information from outside the training dataset is used to create the model, leading to overly optimistic performance metrics and poor generalization to unseen data. In medical imaging, this can happen if labels are inferred from metadata rather than the image itself or same patient in the test are found in the training.

Radiology: A branch of medicine that uses imaging techniques, such as X-rays, CT scans, and MRIs, to diagnose and treat diseases and injuries. In the context of chest X-rays, radiologists interpret images to detect abnormalities in the lungs, heart, and surrounding structures.

11. References

- Brale. (2021). *If vanishing gradients are NOT the problem that ResNets solve, then what is the explanation behind ResNet success?* [Comment] Stack Exchange. <https://ai.stackexchange.com/questions/18703/if-vanishing-gradients-are-not-the-problem-that-resnets-solve-then-what-is-the>
- Gomede, E. (2023). *The ROC Curve: Application and Interpretation in the Health Context*. Medium. <https://medium.com/@evertongomede/the-roc-curve-application-and-interpretation-in-the-health-context-cc06af05a6ec>
- Harun. (2019). *Metrics - Precision, Recall, F1 Score*. Github. <https://hrngok.github.io/posts/metrics3/>
- Holbrook, R. (2024). *Overfitting and Underfitting*. Kaggle. <https://www.kaggle.com/code/ryanholbrook/overfitting-and-underfitting>
- Mariappan, V. (2017). *How to calculate gradients in ResNet architecture?* [Comment] Stack Overflow. <https://stackoverflow.com/questions/44512126/how-to-calculate-gradients-in-resnet-architecture>
- Nbro. (2020). *Why do ResNets avoid the vanishing gradient problem?* [Comment] Stack Exchange. <https://ai.stackexchange.com/questions/17764/why-do-resnets-avoid-the-vanishing-gradient-problem>
- Phung, D., & Le, T. (2024). *Week 03: Convolutional Neural Networks* [Lecture notes]. Site. <https://learning.monash.edu/>
- Phung, D., & Le, T. (2024). *Week 04: Back-Propagation and Optimization for Deep Learning* [Lecture notes]. Site. <https://learning.monash.edu/>
- Phung, D., & Le, T. (2024). *Week 05: Practical Skills in Deep Learning* [Lecture notes]. Site. <https://learning.monash.edu/>
- Phung, D., & Le, T. (2024). *Week 06: Advanced Convolution Neural Networks* [Lecture notes]. Site. <https://learning.monash.edu/>
- Shimao. (2018). *How does backpropagation in residual network works?* [Comment] Stack Exchange. <https://stats.stackexchange.com/questions/370865/how-does-backpropagation-in-residual-network-works>
- Singha, A. (2023). *Breaking the Gradient Barrier: How ResNet Rescued Deep Learning from the Vanishing Gradient Problem*. Medium. https://medium.com/@singhamit_/breaking-the-gradient-barrier-how-resnet-rescued-deep-learning-from-the-vanishing-gradient-problem-ffc7f33eae1a
- Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M., & Summers, R. M. (2017). *ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases*. arXiv. <https://doi.org/10.48550/arXiv.1705.02315>
- Weraduwa, S. (2024). *Understanding ResNet-50: Solving the Vanishing Gradient Problem with Skip Connections*. Medium. <https://medium.com/@sandushiw98/understanding-resnet-50-solving-the-vanishing-gradient-problem-with-skip-connections-5591fcb7ff74>
- Wikipedia. (2025). *Residual neural network, Backward propagation*. https://en.wikipedia.org/wiki/Residual_neural_network#Backward_propagation

Wyszynski, R., & Struniawski, K. (2023). *Residual Neural Networks in Single Instance-Driven Identification of Fungal Pathogens*. Research Gate. <https://doi.org/10.22630/MGV.2023.32.3.3>

Zhang, A., & Lipton, C. Z. (2023). *Dive Into Deep Learning* (1st ed). Cambridge University Press.